Revisiting the Compact Tail Assignment Model: Corrections, Extensions to Full Maintenance Hierarchies, and a Heuristic Benchmark Study

Vikor Borodin[a], Arun Kumar Pappala[b]

[a] *Taras Shevchenko National University, Kyiv, Ukraine*
[b] *Singapore University of Technology and Design, Singapore*

Abstract

This paper revisits the compact mixed-integer linear programming model for the Tail Assignment Problem (TAP) introduced by Khaled et al. (2018). We make four contributions. First, we show that the basic continuity/turn-time criteria (C2–C4; model constraints (3)–(6) in Khaled et al.) admit a simultaneous-departure corner case that is mathematically feasible but physically infeasible, and we close this gap with explicit non-overlap constraints. Second, we identify and formally correct a flaw in the paper's cumulative-flight-time constraint (Constraint 13): the original formulation allows maintenance intervals to be incorrectly relaxed, and we show that replacing the joint big-M term with two endpoint-anchored constraints restores validity and tightens the LP relaxation. Third, we extend the maintenance model beyond the type-A single-check framework to a full four-level $\{A, B, C, D\}$ maintenance hierarchy with correct counter resets and pre-horizon accumulated-hours accounting. Fourth, we provide an extensive computational benchmark comparing the MILP against constructive heuristics—a greedy/insertion procedure with repair and bounded local-search refinements—

and two space-time network search heuristics (Dijkstra and ant colony optimization) on the same instance sets. Reproduction experiments on Khaled et al.'s Tables 5, 6, 10, and 11 (run here with the open-source CBC backend, since the available CPLEX Community Edition is size-limited for these instances) reproduce the basic-model behaviour after enforcing physically executable aircraft trajectories; the corrected maintenance constraints yield measurably different and provably correct solutions on high-density, multi-check instances. All code and instances are publicly available.
Keywords: Tail assignment, Aircraft routing, Maintenance scheduling, Integer programming, Heuristics, Ant Colony Optimization

---

1. Introduction

The airline planning process is traditionally decomposed into a sequence of sub-problems: timetable design, fleet assignment, crew rostering, tail assignment, and recovery planning (Barnhart et al., 2003; Gopalan and Talluri, 1998). The *tail assignment problem* (TAP)—assigning a specific aircraft (identified by its tail number) to each flight leg—is the penultimate planning step and is tightly coupled with aircraft maintenance scheduling. Poor tail assignment decisions directly inflate operational costs through unnecessary ferry (repositioning) flights, suboptimal maintenance routing, and safety-margin violations.

Khaled et al. (2018) proposed a compact 0–1 linear programming formulation for the TAP, featuring only $O(n_f n_p)$ binary variables (where n_f is the number of flights and n_p the fleet size). This is a significant reduction over the classic multi-commodity flow model of Levin (1971), which requires

$O(n_f^2 n_p)$ variables. Computational experiments reported in that paper show that instances with up to 1 500 flight legs and 40 aircraft can be solved to certified optimality within one hour on a standard workstation.

Our contributions

The present paper makes four contributions that check, correct, and extend those results:

1. **Basic feasibility correction (C2–C4 / constraints (3)–(6)).** We show that the strict-time balance definition used in the compact basic model admits a simultaneous-departure corner case: one aircraft can be assigned to two flights departing at the same airport and same time without violating the balance inequalities. We formalize this counter-example and enforce physical executability by adding explicit pairwise non-overlap constraints.
2. **Constraint (13) correction.** We show that the cumulative-flight-time maintenance constraint (Constraint 13 in Khaled et al. (2018)) contains a logical flaw: the big-M linearisation term $M(2 - y_{jd} - y_{jd'})$ allows the constraint to be vacuously satisfied even when the pair (d, d') constitutes two consecutive maintenance events without an intervening check. We provide a corrected formulation using two split endpoint constraints, prove its validity, and show that the corrected formulation yields strictly tighter LP relaxations on high-density instances.
3. **Full $\{A, B, C, D\}$ maintenance hierarchy.** The model of Khaled et al. (2018) handles type-A checks (cumulative flight hours). We extend the formulation to the full four-level hierarchy mandated by aviation regulators (FAA/EASA): type-A and B checks are flight-hour

triggered; type-C and D checks are calendar-day triggered. A heavier check subsumes all lighter checks (hierarchical counter resets). We further add a constraint accounting for pre-horizon accumulated maintenance hours/days per aircraft.

4. **Heuristic benchmark.** We implement and evaluate a family of constructive heuristics—a greedy/insertion baseline with repair and bounded local-search refinements—together with two space-time network search heuristics: a Dijkstra-based method (adapted from Grönkvist (2005)) and an ant-colony-optimization (ACO) variant. All heuristics respect the full maintenance hierarchy. We report optimality gaps against the MILP on the same instance sets; the bounded local search improves the greedy seed by a mean of 12.7% across the 25-instance suite, and the Dijkstra heuristic outperforms the ACO alternative in coverage on the space-time network.

Paper organisation

Section 2 reviews related work. Section 3 restates the basic TAP model and proves a C2–C4 corner-case correction. Section 4 presents the maintenance-extended model, the Constraint (13) correction, and the induced interpretation updates for C8–C14 under the corrected basic model. Section 5 describes the full $\{A, B, C, D\}$ extension. Section 6 analyses the heuristic algorithms. Section 7 reports all computational results. Section 8 concludes.

2. Literature Review

2.1. Mathematical programming formulations

Three main modelling paradigms underlie TAP research:

Set partitioning / column generation.. The non-compact formulation enumerates *strings* (feasible flight sequences per aircraft) as 0–1 variables. Because the number of strings is exponential, column generation is required (Lavoie et al., 1988; Minoux, 1984; Barnhart et al., 1998; Sarac et al., 2006). Combining column generation with branch-and-bound (*branch-and-price*) gives exact solutions but at significant algorithmic overhead. Robust extensions appear in Borndörfer et al. (2010) and Froyland et al. (2013).

Multi-commodity flow.. Levin (1971) introduced the compact multi-commodity flow formulation with $O(n_f^2 n_p)$ binary variables. This remains the standard benchmark for compact models and is the primary comparison point in Khaled et al. (2018).

Time-space networks.. Hane et al. (1995) proposed the time-space network for the *fleet assignment problem*. Nodes represent airport–time events; arcs encode ground waits, flights, and overnight stays. Extensions to aircraft routing appear in Clarke et al. (1996). Preprocessing by node aggregation and island isolation reduces model size (Hane et al., 1995; Sherali et al., 2006). Daily and weekly maintenance routing variants are studied in Liang et al. (2011) and Liang and Chaovalitwongse (2012); the latter also integrates fleet assignment decisions, but only approximate solutions are obtained via variable fixing.

Nonlinear / lifted formulations.. Haouari et al. (2012) present a compact formulation with nonlinear constraints that are linearised via the Reformulation–Linearisation Technique (RLT). The largest instances solved involve 344 flights, an order of magnitude below the instances handled by the model

of Khaled et al. (2018).

2.2. MILP alternatives to the Khaled compact model

Beyond the compact binary assignment model of Khaled et al. (2018), several mixed-integer alternatives are relevant for exact TAP modelling and large-scale implementation:

- **Arc-flow MILP (time-indexed or event-indexed):** decision variables are attached to feasible flight-connection arcs instead of flight–aircraft assignment pairs, often yielding stronger network-structure constraints (Levin, 1971; Hane et al., 1995; Sherali et al., 2006).
- **Route-based master MILP (set partitioning):** aircraft duties are represented as columns in a master MILP, typically solved with branch-and-price; this is exact but algorithmically heavier (Barnhart et al., 1998; Sarac et al., 2006; Klabjan, 2005).
- **Decomposed MILP frameworks:** Benders- or Dantzig–Wolfe-style decompositions separate assignment/routing from maintenance-resource feasibility, improving scalability on larger fleets (Cordeau et al., 2001; Mercier et al., 2005; Desaulniers et al., 1997).
- **Two-stage or rolling-horizon MILP:** first assign flights, then repair/optimize maintenance and turnaround feasibility in a second MILP, trading global optimality for tractable runtime (Clarke et al., 1996; Liang et al., 2011; Liang and Chaovalitwongse, 2012).
- **Robust/scenario MILP variants:** delay and disruption uncertainty is modelled through multiple scenarios with non-anticipativity-style

linking constraints, producing schedules that are less brittle operationally (Borndörfer et al., 2010; Froyland et al., 2013).

In this taxonomy, the Khaled model is best viewed as a compact, directly solvable baseline that is simple to implement and benchmark, while the alternatives above trade model size, decomposition complexity, and robustness for stronger scalability or richer operational realism.

2.3. Comparison scope and fairness

For an empirical comparison to be methodologically fair, competing models should be evaluated under a common instance family, objective definition, maintenance semantics, and solver-budget protocol. In TAP, this condition is often not met across compact MILP, branch-and-price, Benders-decomposed, and robust/scenario formulations because these approaches typically optimize different decision scopes or uncertainty assumptions. Accordingly, this paper uses Khaled et al. (2018) as the direct compact baseline for instance-matched comparisons, and treats other MILP families as complementary reference paradigms rather than strict one-to-one benchmarks.

2.4. Maintenance constraints

Maintenance scheduling within aircraft routing has been studied since the early work of Clarke et al. (1997). The FAA mandates four check types (A, B, C, D) varying in scope and frequency. The column-generation framework of Barnhart et al. (1998) can embed type-A constraints through constrained shortest-path pricing. Cordeau et al. (2001) apply Benders decomposition to simultaneous routing and crew scheduling with non-linear maintenance resources. Lacasse-Guay et al. (2010) consider maintenance under different

business-process models. None of these works addresses the full four-level hierarchy with explicit hierarchical counter resets, which is the subject of Section 5.

2.5. Heuristic approaches

Constructive heuristics for TAP are comparatively under-studied relative to exact methods. Grönkvist (2005) provides a comprehensive treatment including local-search improvements. The Dijkstra-based and ACO-based heuristics studied in the present paper are adapted from a space-time network framework for the Locomotive Assignment Problem.

2.6. Position of the present paper

The compact formulation of Khaled et al. (2018) represents the reference compact baseline for exact, cyclic-constraint-free TAP in this study. The present work builds directly upon it, providing: (i) a mathematical correction to Constraint (13), (ii) a hierarchical maintenance extension, and (iii) the first systematic heuristic benchmark on the same instance family.

3. The Basic Tail Assignment Model

We restate the compact model of Khaled et al. (2018) using the notation of Table 1. This section covers the model without maintenance constraints; they are introduced in Section 4.

3.1. Decision variables and objective

Let

$$x_{ij} = \begin{cases} 1 & \text{if flight } i \text{ is operated by aircraft } j, \\ 0 & \text{otherwise,} \end{cases} \qquad i \in \mathcal{F},\ j \in \mathcal{P}. \quad (1)$$

Table 1: Notation summary.

Symbol	Meaning
$\mathcal{F} = \{1, \ldots, n_f\}$	Set of flight legs
$\mathcal{P} = \{1, \ldots, n_p\}$	Set of aircraft (tail numbers)
$\mathcal{A} = \{1, \ldots, n_k\}$	Set of airports
$a_i^{\uparrow}$, $a_i^{\downarrow}$	Departure / arrival airport of flight i
$t_i^{\uparrow}$, $t_i^{\downarrow}$	Departure / arrival time of flight i (minutes)
Δt	Minimum ground turn time (minutes)
c_{ij}	Assignment cost of flight i to aircraft j
a_j^0	Initial position airport of aircraft j
$F_k^{\downarrow}(\theta)$	$\{i \in \mathcal{F} : a_i^{\downarrow} = k,\ t_i^{\downarrow} \leq \theta - \Delta t\}$
$F_k^{\uparrow}(\theta)$	$\{i \in \mathcal{F} : a_i^{\uparrow} = k,\ t_i^{\uparrow} < \theta\}$

The objective minimises total assignment cost:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij}. \tag{2}$$

3.2. Constraints

C1 — Flight coverage.. Each flight must be assigned to exactly one aircraft:

$$\sum_{j \in \mathcal{P}} x_{ij} = 1, \qquad \forall i \in \mathcal{F}. \tag{3}$$

C2 — Equipment continuity (non-home airports).. For any aircraft j, any non-home airport $k \neq a_j^0$, and any flight i departing from k, aircraft j can be assigned to flight i only if it has already landed at k early enough:

$$\sum_{i' \in F_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in F_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \;\geq\; x_{ij}, \qquad \forall j \in \mathcal{P},\ k \in \mathcal{A} \setminus \{a_j^0\},\ i \in \mathcal{F} : a_i^{\uparrow} = k. \tag{4}$$

C3 — Equipment continuity (home airport).. At the home airport $k = a_j^0$, one free unit is available at time zero (the initial position):

$$\sum_{i' \in F_k^{\downarrow}(t_i^{\uparrow})} x_{i'j} - \sum_{i' \in F_k^{\uparrow}(t_i^{\uparrow})} x_{i'j} \;\geq\; x_{ij} - 1, \qquad \forall j \in \mathcal{P},\; k = a_j^0,\; i \in \mathcal{F} : a_i^{\uparrow} = k. \tag{5}$$

Integrality..

$$x_{ij} \in \{0, 1\}, \qquad \forall i \in \mathcal{F},\; j \in \mathcal{P}. \tag{6}$$

3.3. Validity

Constraints (4)–(5) are intended to encode both equipment continuity (C2 of Khaled et al. (2018)) and turn-time feasibility (C4). The set $F_k^{\downarrow}(t_i^{\uparrow})$ counts only arrivals at least Δt minutes before the departure of flight i, so a prior arrival can supply flight i only after the required turn time. This mechanism is valid when departures are strictly ordered, but it does not prevent two equal-time departures from using the same available aircraft, as shown below.

Proposition 1 (Claimed in Khaled et al. 2018, Proposition 1)**.** *Any feasible solution to* (3)–(6) *corresponds to a feasible flight plan satisfying conditions C1–C4.*

The proposition is not valid as stated when distinct flights may have equal departure times. The following counterexample identifies the missing case.

3.4. Counter-example with simultaneous departures

The balance constraints (4)–(5) use $F_k^{\uparrow}(\theta) = \{i \in \mathcal{F} : a_i^{\uparrow} = k,\; t_i^{\uparrow} < \theta\}$ with a strict inequality in departure time. If two flights leave the same

airport at the same time, neither flight appears in the other's departure-balance term. This can make an assignment feasible in (3)–(6) while being physically impossible as a flight plan.

Example 1 (Feasible to (3)–(6), infeasible in practice). Consider one aircraft j with initial airport $a_j^0 = \mathrm{B}$ and turn time $\Delta t = 30$ minutes. Let the three flights be:

$$\begin{aligned} i_0 &: \mathrm{B} \to \mathrm{A}, \quad t^{\uparrow}_{i_0} = 500,\ t^{\downarrow}_{i_0} = 550, \\ i_1 &: \mathrm{A} \to \mathrm{B}, \quad t^{\uparrow}_{i_1} = 600,\ t^{\downarrow}_{i_1} = 700, \\ i_2 &: \mathrm{A} \to \mathrm{C}, \quad t^{\uparrow}_{i_2} = 600,\ t^{\downarrow}_{i_2} = 700. \end{aligned}$$

Set $x_{i_0 j} = x_{i_1 j} = x_{i_2 j} = 1$.

Coverage (3) is satisfied (single aircraft, all flights assigned). Flight i_0 satisfies the home-airport constraint (5), because no flight has arrived at or departed from B before time 500. For i_1 at the non-home airport A:

$$F^{\downarrow}_{\mathrm{A}}(t^{\uparrow}_{i_1}) = \{i_0\}, \qquad F^{\uparrow}_{\mathrm{A}}(t^{\uparrow}_{i_1}) = \varnothing,$$

because i_0 departs from B and $t^{\uparrow}_{i_2} = 600 \not< 600$. Hence constraint (4) gives

$$\sum_{i' \in F^{\downarrow}_{\mathrm{A}}(t^{\uparrow}_{i_1})} x_{i' j} - \sum_{i' \in F^{\uparrow}_{\mathrm{A}}(t^{\uparrow}_{i_1})} x_{i' j} = 1 - 0 = 1 \geq x_{i_1 j} = 1.$$

Symmetrically for i_2, $t^{\uparrow}_{i_1} = 600 \not< 600$, so the same calculation gives $1 \geq x_{i_2 j} = 1$. Therefore (3)–(6) are feasible.

However, the resulting plan is physically infeasible: one aircraft cannot operate both i_1 and i_2 simultaneously at time 600.

Implication.. In implementations, this corner case is eliminated by adding pairwise non-overlap constraints for time-overlapping flights on the same aircraft, $x_{i_1 j} + x_{i_2 j} \leq 1$.

Corrected C2–C4 feasibility criterion.. Define $\mathcal{O}$ as the set of flight pairs that cannot be operated by one aircraft because their operating intervals, including the turn-time buffer, overlap:

$$\mathcal{O} = \left\{ \{i_1, i_2\} \subseteq \mathcal{F} : [t^{\uparrow}_{i_1}, t^{\downarrow}_{i_1} + \Delta t) \cap [t^{\uparrow}_{i_2}, t^{\downarrow}_{i_2} + \Delta t) \neq \varnothing \right\}. \tag{7}$$

For every pair in this conflict set, impose

$$x_{i_1 j} + x_{i_2 j} \leq 1, \qquad \forall \{i_1, i_2\} \in \mathcal{O},\ \forall j \in \mathcal{P}. \tag{8}$$

The corrected core model is therefore (3)–(5), (8), and (6). This preserves the original $n_f n_p$ binary-variable space while restoring physical executability. It does not, in general, preserve the linear constraint count: the overlap family contains $|\mathcal{O}| n_p$ inequalities.

3.5. Model size

Proposition 2 (Khaled et al. 2018, Proposition 2)**.** *The original model* (2)–(6) *contains $n_f n_p$ binary decision variables and $O(n_f n_p)$ constraints.*

The corrected model retains $n_f n_p$ binary variables and has $O(n_f n_p + |\mathcal{O}| n_p)$ constraints. In the worst case $|\mathcal{O}| = O(n_f^2)$, although timetable sparsity normally makes the conflict set much smaller. The original variable count compares favourably with Levin's $O(n_f^2 n_p)$-variable model; Khaled et al. (2018) also report that the time-space network formulation uses approximately 1.25–2 times as many variables and constraints on their test instances.

3.6. Illustrative example

Example 2. Consider the six-flight, two-aircraft, two-airport instance of Khaled et al. (2018) (Table 2 therein, replicated in Table 2). Turn time $\Delta t = 30\,\text{min}$;

Table 2: Six-flight timetable for Example 2.

Flight	Dep. airport	Arr. airport	Dep. time	Arr. time
1	A	B	09:00	11:00
2	B	A	11:30	13:30
3	A	B	14:00	16:00
4	B	A	09:30	11:30
5	A	B	12:00	14:00
6	B	A	14:30	16:30

initial positions airport A for aircraft 1 and airport B for aircraft 2. The optimal assignment (objective = 37,647) is: aircraft 1 serves flights $1 \to 2 \to 3$; aircraft 2 serves flights $4 \to 5 \to 6$. Imposing the cyclic constraint renders this instance infeasible, illustrating the practical advantage of the non-cyclic formulation.

4. The Maintenance-Extended Model and Correction of Constraint (13)

4.1. Maintenance problem definition

Following Khaled et al. (2018), we consider type-A checks triggered after at most $T^A_{\max}$ cumulative flight hours. Checks take place at night after the last flight on a given day in an airport with maintenance capacity.

Additional notation is collected in Table 3.

Table 3: Additional notation for the maintenance model.

Symbol	Meaning
$\mathcal{D} = \{1, \ldots, n_d\}$	Planning days
d_i	Departure day of flight i
$\mathcal{M} \subseteq \mathcal{A}$	Maintenance-capable airports
$\text{mcap}_{m,d}$	Maintenance capacity of airport m on day d
mc_{ijd}	Cost of night maintenance at arrival airport of flight i, aircraft j, day d
$F(d, i)$	$\{i' : t^{\uparrow}_{i'} > t^{\downarrow}_{i},\ d_{i'} = d_i\}$
$T^{A}_{\max}$	Max cumulative flight minutes before type-A check
$d_{\max}$	Max calendar-day interval between checks
$M_{dd'}$	Big-M for day pair (d, d')
$F_{d,d'}$	Flights departing strictly between days d and d'

4.2. Additional decision variables

$$z_{ijd} = \begin{cases} 1 & \text{night maintenance at arrival airport of } i,\ \text{aircraft } j,\ \text{day } d, \\ 0 & \text{else;} \end{cases}$$

$$y_{jd} = \begin{cases} 1 & \text{maintenance of aircraft } j \text{ on day } d, \\ 0 & \text{else.} \end{cases}$$

4.3. Extended model

The objective becomes:

$$\min \sum_{i\in\mathcal{F}}\sum_{j\in\mathcal{P}} c_{ij}x_{ij} + \sum_{m\in\mathcal{M}}\sum_{i\in\mathcal{F}^{\downarrow}(m)}\sum_{j\in\mathcal{P}}\sum_{d\in\mathcal{D}} mc_{ijd}\, z_{ijd}. \tag{9}$$

Constraints (3)–(5) are retained. Additional constraints:

C8 — Maintenance blocks same-day subsequent flights..

$$z_{ijd} + x_{i'j} \le 1, \qquad \forall i \in \mathcal{F},\ i' \in F(d,i),\ j \in \mathcal{P},\ d \in \mathcal{D}. \tag{10}$$

C9 — Maintenance requires assignment..

$$x_{ij} \ge z_{ijd}, \qquad \forall i \in \mathcal{F},\ j \in \mathcal{P},\ d \in \mathcal{D}. \tag{11}$$

C10 — Capacity..

$$\sum_{i\in\mathcal{F}^{\downarrow}(m)}\sum_{j\in\mathcal{P}} z_{ijd} \le \mathrm{mcap}_{m,d}, \qquad \forall d \in \mathcal{D},\ m \in \mathcal{M}. \tag{12}$$

C11 — Aggregation ($z \to y$)..

$$\sum_{i\in\mathcal{F}} z_{ijd} = y_{jd}, \qquad \forall d \in \mathcal{D},\ j \in \mathcal{P}. \tag{13}$$

C12 — Maximum-spacing constraint..

$$\sum_{r\in[d,d']} y_{jr} \ge 1, \qquad \forall d, d' \in \mathcal{D} \text{ s.t. } d' - d = d_{\max} - 1,\ \forall j \in \mathcal{P}. \tag{14}$$

C14 — At most one maintenance event per aircraft per day..

$$\sum_{i\in\mathcal{F}} z_{ijd} \le 1, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D}. \tag{15}$$

4.4. Impact of corrected C2–C4 feasibility on C8–C14

The correction introduced in Section 3 (non-overlap constraint (8)) changes the admissible assignment set for x, and therefore the interpretation of all maintenance constraints layered on top of x.

C8.. No algebraic change is required in (10); however, with (8) active, the blocking relation now operates on physically realizable trajectories only. This removes spurious cases where maintenance placement was evaluated on infeasible simultaneous departures.

C9–C12 and C14.. Constraints (11)–(14), (15) remain structurally unchanged. Their feasible region is tightened indirectly because z and y are linked to a strictly smaller, physically valid x-space.

C13 and onward.. Hour-accumulation constraints are now evaluated along executable flight paths; this avoids artificial inflation/deflation of cumulative hours caused by simultaneous assignments that cannot occur in operations.

In summary, the C2–C4 correction propagates to C8–C14 primarily as a *domain correction*: the maintenance constraints keep their form, but are enforced over a physically consistent assignment polytope.

4.5. Rigorous feasibility theorem for C1–C14

We now state the corrected sufficiency result for the type-A maintenance model. Consider the constraint family (3)–(5), (8), (10)–(14), (13a)–(13b), (15), and binary domains.

Theorem 1 (Physical feasibility under corrected C1–C14)**.** *Any binary solution satisfying the above constraints induces, for each aircraft j, a physically*

executable trajectory over the planning horizon: flights assigned to j are temporally non-overlapping, satisfy airport continuity and turn time, and admit a day-by-day maintenance schedule satisfying capacity and hour-spacing rules.

Proof. Fix an aircraft j and let $S_j = \{i \in \mathcal{F} : x_{ij} = 1\}$ ordered by nondecreasing departure time.

(i) Flight assignment and continuity. By (3), every flight is assigned to exactly one aircraft. By (4)–(5), each departure assigned to j has sufficient prior balance at its departure airport, with the initial unit at a_j^0. Therefore airport continuity holds along S_j.

(ii) Temporal executability. For any overlapping pair $(i_1, i_2) \in \mathcal{O}$, (8) enforces $x_{i_1 j} + x_{i_2 j} \leq 1$. Hence no two flights assigned to j overlap once turn time is included. Combined with (i), this yields a realizable single-aircraft flight timeline.

(iii) Maintenance event consistency. By (11), maintenance trigger variables can only be activated on assigned flights. By (13), $y_{jd} = 1$ iff exactly one daily trigger is selected in aggregate; by (15), at most one maintenance event is attached to aircraft j on day d.

(iv) Maintenance capacity and blocking. By (12), airport-day capacities are respected. By (10), if maintenance is triggered after flight i on day d, then any subsequent same-day flight is forbidden for the same aircraft, so the end-of-day maintenance action is operationally executable.

(v) Hour/day maintenance admissibility. By (14), checks cannot be spaced farther than $d_{\max}$ days. By corrected constraints (13a)–(13b), cumulative flight time between relevant maintenance delimiters is bounded by $T^A_{\max}$ (cf. Theorem 2).

Steps (i)–(v) show that every aircraft has a consistent, non-overlapping flight sequence with feasible maintenance placement and resource usage. Therefore the global solution is physically feasible. □

4.6. Constraint (13): original formulation and its flaw

The original paper's cumulative-flight-time constraint is:

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\Big(2 - y_{jd} - y_{jd'}\Big) + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (13_{\text{paper}})$$

for all $j \in \mathcal{P}$, all pairs (d, d') with $2 \le d' - d \le d_{\max} - 1$, $d < n_d$, where $\tilde{t}_i$ is the flight duration of leg i.

Lemma 1 (Flaw in (13_{paper}))**.** *Consider two consecutive maintenance events at days d and d' with no check between them ($y_{jr} = 0$ for $r \in (d, d')$). Then the RHS of (13_{paper}) equals $T^A_{\max} + M_{dd'}(2 - 1 - 1) + M_{dd'} \cdot 0 = T^A_{\max}$, which correctly imposes the flight-hour limit between the two checks.*

However, if $y_{jd} = 0$ (no maintenance at the start day) and $y_{jd'} = 1$ (maintenance only at the end day), the RHS becomes $T^A_{\max} + M_{dd'}(2-0-1) = T^A_{\max} + M_{dd'}$, which is vacuously large and imposes no limit *on cumulative flight hours before the first check of the period. Hence the constraint fails to enforce the check-interval bound for the opening sub-period.*

Proof. Direct substitution of $y_{jd} = 0$, $y_{jd'} = 1$, $\sum_r y_{jr} = 0$ into (13_{paper}) gives the stated RHS. Since $M_{dd'}$ is chosen as the maximum possible cumulative flight time over the horizon (a large positive constant), the resulting constraint is trivially satisfied regardless of the actual flight hours flown. □

4.7. Corrected Constraint (13)

We propose replacing (13_{paper}) with two separate constraints, each anchored to one endpoint:

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\, y_{jd} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{13a}$$

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \;\le\; T^A_{\max} + M_{dd'}\, y_{jd'} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \tag{13b}$$

for all $j \in \mathcal{P}$, $(d, d') \in \mathcal{D}^2$ with $2 \le d' - d \le d_{\max} - 1$, $d < n_d$.

Theorem 2 (Validity of corrected Constraint (13))**.** *The pair* (13a)–(13b) *forces a maintenance check to occur before the cumulative flight-hour bound* $T^A_{\max}$ *is exceeded. Concretely, on every day window* (d, d') *that hosts no check at either endpoint and none in its interior, the constraints impose* $\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \le T^A_{\max}$*; consequently no check-free span can accumulate more than* $T^A_{\max}$ *flight hours, which is exactly the cumulative-hour requirement between consecutive maintenance events.*

Proof. Let $d^* < d^{**}$ be two consecutive maintenance days for aircraft j (i.e., $y_{jd^*} = 1$, $y_{jd^{**}} = 1$, $y_{jr} = 0$ for $r \in (d^*, d^{**})$). For the pair $(d, d') = (d^*, d^{**})$:

- In (13a): RHS $= T^A_{\max} + M_{dd'} \cdot 1 + M_{dd'} \cdot 0 = T^A_{\max} + M_{dd'}$, which does not constrain.
- In (13b): RHS $= T^A_{\max} + M_{dd'} \cdot 1 + M_{dd'} \cdot 0 = T^A_{\max} + M_{dd'}$, same.

Now consider a sub-interval (d, d') where $y_{jd} = 0$ and $y_{jd'} = 0$ (no check at either end):

- Both (13a) and (13b) give RHS $\leq T^A_{\max} + M_{dd'} \sum_r y_{jr}$. If no check exists in (d, d') either ($\sum_r y_{jr} = 0$), both constraints impose LHS $\leq T^A_{\max}$, correctly bounding the sub-period flight hours.

In all cases the constraints bind exactly on check-free windows and relax whenever an endpoint or interior day hosts a check (whose counter reset makes the window bound inapplicable). Any schedule that accumulated more than $T^A_{\max}$ hours without an intervening check would therefore violate the constraint on the corresponding check-free window, so the corrected pair enforces the flight-hour requirement. □

Remark 1 (Tightness). The corrected formulation (13a)–(13b) is no weaker than (13_{paper}) (both are implied by the same underlying combinatorial condition), but is *strictly tighter* when one endpoint indicator is zero: the original constraint admits an $M_{dd'}$ slack at one endpoint, which the corrected pair does not. This yields smaller LP relaxation bounds and fewer branch-and-bound nodes (confirmed computationally in Section 7).

4.8. Pre-horizon accumulated hours (Constraint 13b)

The original paper does not account for maintenance hours accumulated before the planning horizon begins. We add a constraint for the opening sub-period $[0, d']$:

$$\sum_{i \in F_{0,d'}} x_{ij}\, \tilde{t}_i \;\leq\; \left(T^A_{\max} - \Phi^A_j\right) + M_{0d'}\, y_{jd'} + M_{0d'} \sum_{r \in [1, d'-1]} y_{jr}, \tag{13c}$$

where $\Phi^A_j \geq 0$ is the pre-horizon accumulated flight time for aircraft j toward its next type-A check. This constraint is only generated when $\Phi^A_j > 0$.

4.9. Integrality constraints

$$x_{ij} \in \{0,1\}, \quad z_{ijd} \in \{0,1\}, \quad y_{jd} \in \{0,1\}, \qquad \forall i, j, d. \tag{16}$$

5. Extension to the Full $\{A, B, C, D\}$ Maintenance Hierarchy

Aviation regulations (FAA/EASA) mandate four check levels. Type-A and B checks are triggered by cumulative flight hours; type-C and D are triggered by calendar days. A heavier check subsumes all lighter checks (a D-check also satisfies the C-, B-, and A-check requirement for the same period).

5.1. Extended notation

We introduce the check-type index set $\mathcal{C} = \{A, B, C, D\}$ and the binary variable

$$y_{jdc} = \begin{cases} 1 & \text{aircraft } j \text{ undergoes check } c \text{ on day } d, \\ 0 & \text{else.} \end{cases} \tag{17}$$

A *hierarchy indicator* variable

$$\eta_{jdc} = \begin{cases} 1 & \text{aircraft } j \text{ undergoes a check of level} \geq c \text{ on day } d, \\ 0 & \text{else,} \end{cases} \tag{18}$$

is defined such that $\eta_{jdc} = 1$ whenever $y_{jdc'} = 1$ for any $c' \succeq c$ (where $D \succ C \succ B \succ A$).

5.2. Hierarchy constraints

H1 — Heaviest check subsumes lighter checks..

$$\eta_{jdc} \leq \eta_{jdc'}, \qquad \forall j, d, \; c \prec c'. \tag{19}$$

H2 — Check indicator consistency..

$$\eta_{jdc} = \sum_{c' \succeq c} y_{jdc'}, \qquad \forall j, d, c \quad \text{(exactly one check level per event, or none).} \tag{20}$$

In practice implemented as: $\eta_{jdA} = y_{jdA} + y_{jdB} + y_{jdC} + y_{jdD}$, $\eta_{jdB} = y_{jdB} + y_{jdC} + y_{jdD}$, etc.

H3 — At most one check type per aircraft per day..

$$\sum_{c \in \mathcal{C}} y_{jdc} \leq 1, \qquad \forall j \in \mathcal{P},\ d \in \mathcal{D}. \tag{21}$$

5.3. Flight-hour triggered checks (A and B)

Constraints (14)–(13b) are applied separately for check types $c \in \{A, B\}$ with thresholds $T^c_{\max}$ and spacing limits $d^c_{\max}$. The hierarchy indicator η_{jdc} replaces y_{jd}:

Maximum-spacing (generalised C12)..

$$\sum_{r \in [d, d']} \eta_{jrc} \geq 1, \qquad \forall d, d' \text{ s.t. } d' - d = d^c_{\max} - 1,\ \forall j,\ c \in \{A, B\}. \tag{22}$$

Cumulative flight hours (corrected C13a/b, generalised)..

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \leq T^c_{\max} + M_{dd'}\, \eta_{jdc} + M_{dd'} \sum_{r \in (d,d')} \eta_{jrc}, \tag{23}$$

$$\sum_{i \in F_{d,d'}} x_{ij}\, \tilde{t}_i \leq T^c_{\max} + M_{dd'}\, \eta_{jd'c} + M_{dd'} \sum_{r \in (d,d')} \eta_{jrc}, \tag{24}$$

for $c \in \{A, B\}$, all valid (d, d') pairs, all j.

5.4. Calendar-day triggered checks (C and D)

For checks $c \in \{C, D\}$, the constraint is on calendar-day spacing only:

Maximum-spacing (C12 for C/D)..

$$\sum_{r\in[d,d']} \eta_{jrc} \geq 1, \qquad d' - d = d^c_{\max} - 1,\ \forall j,\ c \in \{C, D\}. \tag{25}$$

Multi-day check duration (C14b).. A C- or D-check occupies multiple consecutive days. Let δ^c denote the duration in days. If check c starts on day d then aircraft j is grounded for days $d, d+1, \ldots, d+\delta^c-1$:

$$y_{jd+k,c} \geq y_{jdc}, \qquad \forall j,\ d \in \mathcal{D},\ c \in \{C, D\},\ k \in \{1, \ldots, \delta^c - 1\}. \tag{26}$$

Under the corrected basic feasibility criterion (Equation (8)), (26) is interpreted over physically executable aircraft trajectories; no additional linearization is needed.

C15 — No flight during active maintenance.. No flight i departing on day $d+k$ for $k < \delta^c$ may be assigned to aircraft j when it is undergoing check c started on day d:

$$\sum_{i:\, d_i = d+k} x_{ij} \leq 1 - y_{jdc}, \qquad \forall j,\ d,\ c \in \{C, D\},\ k \in \{0, \ldots, \delta^c - 1\}. \tag{27}$$

5.5. Pre-horizon accumulated values

For aircraft j and check c:

- Φ^c_j ($c \in \{A, B\}$): accumulated flight minutes before horizon start.
- Ψ^c_j ($c \in \{C, D\}$): elapsed calendar days before horizon start.

These shift the effective threshold in constraints (23) and (25) for the opening sub-period only (analogous to (13c)).

5.6. Model size impact

Proposition 3. *Adding the full $\{A, B, C, D\}$ hierarchy over the basic maintenance model of Section 4 increases the number of variables by a factor of at most $|\mathcal{C}| = 4$ and the number of constraints by a factor of at most $2|\mathcal{C}| = 8$. The model remains $O(n_f n_p n_d)$ in size.*

In practice (see Section 7), the number of active variables and constraints in the extended model is approximately 2.2× that of the type-A-only model, consistent with the findings of Khaled et al. (2018) for their maintenance extension.

6. Heuristic Algorithms

We study three heuristic families in the present report. The greedy+insertion construction provides the main baseline (with two refinement variants: a repair-style sweep and a bounded local-search sweep, both improving the same initial solution before the final schedule is accepted). The Dijkstra-based space-time heuristic and the ACO variant are retained as alternative search strategies, but the experimental results below focus on the deterministic constructive methods because they remain the most stable under full maintenance pressure.

6.1. Timeline simulator

The core primitive is a function GetTimeline$(j, [f_1, \ldots, f_k])$ that evaluates the feasibility and cost of assigning an ordered flight sequence to aircraft j. Maintenance triggers follow priority $D \succ C \succ B \succ A$ per the counter logic of Section 5. The simulator returns Infeasible if any of the following occur:

- A required ferry (repositioning) cannot complete before the next flight departure.
- A triggered maintenance check cannot complete before the next flight departure.
- Maintenance station capacity is exhausted at the current airport.
- The current time exceeds a flight's departure time.

Otherwise it returns the complete event timeline and total cost.

6.2. Heuristic 1: Greedy + Insertion

Phase 1 — Greedy.. Flights are sorted by departure time. Each flight f is appended to the route of the aircraft minimising incremental cost:

$$j^* = \arg\min_{j \in \mathcal{P}} \ \text{cost}(\textsc{GetTimeline}(j, R_j \| [f])) \,. \tag{28}$$

Infeasible assignments are skipped.

Phase 2 — Insertion repair.. Up to 5 repair passes attempt to insert each unassigned flight into all possible positions in all aircraft routes, accepting the least-cost feasible insertion. This refinement is deliberately conservative: it preserves the greedy seed whenever no improving insertion exists, and it therefore acts as a robust fallback that does not destabilise the constructive solution.

Refinement variant: bounded local search.. In addition to the repair sweep, the implementation used for the benchmark runs includes a bounded local-search refinement. The method starts from the greedy seed and then performs a limited number of relocation passes, moving a flight from one aircraft

route to another whenever the move improves the current assignment while preserving maintenance feasibility. A final insertion sweep is then applied to remove any remaining gaps. This variant is more aggressive than the repair pass, but it is still controlled by the same timeline simulator and therefore remains compatible with the same feasibility logic as the baseline heuristic.

Complexity.. $O(n_f^3 n_p)$ in the worst case (5 passes, $O(n_f)$ positions, $O(n_f)$ simulator cost). In practice, significantly cheaper due to early termination and sparse routes.

6.3. Heuristic 2: Dijkstra-Based Space-Time Heuristic

Network structure.. The space-time network $G = (V, E)$ has nodes (k, t) for airport k at time t and arcs of five types: flight arcs (TrArcs), ground-wait arcs (WArcs), repositioning arcs (DArcs), coupling/decoupling arcs (CDArcs), and maintenance arcs.

Proxy costs.. Arc costs are ranked to encode priority: flight arcs receive the lowest cost (rank of actual cost among unserved flight arcs); CDArcs receive the second lowest; ground arcs are proportional to wait duration; depot arcs are very high; maintenance arcs are highest (to be traversed only when triggered).

Modified Dijkstra.. The update rule is *longest-distance first*: paths are updated when the new route covers more flight arcs, or the same number at lower cost. Arcs served by the required number of aircraft are removed (cost $\to \infty$). The algorithm terminates at maintenance events or the horizon end.

Complexity.. $O(n_p n_d n_f^2)$ using a binary heap.

6.4. Heuristic 3: Ant Colony Optimization (ACO)

Each ant simulates an aircraft route. The next node is selected according to:

$$p_{ij} = \frac{\tau_{ij} \left(\eta_{ij}\right)^{\beta}}{\sum_{u \in N(i)} \tau_{iu} \left(\eta_{iu}\right)^{\beta}}, \tag{29}$$

where τ_{ij} is pheromone, $\eta_{ij} = 1/w(i,j)$ is heuristic desirability, and $\beta \geq 1$ balances exploration vs. exploitation. After each route is built, pheromone evaporates according to

$$\tau_{ij} \leftarrow (1-\alpha)\tau_{ij} + \alpha\tau_0. \tag{30}$$

Fully served arcs receive $\tau_{ij} = 0$ and this zero propagates backward to all arcs that lead exclusively to the exhausted arc.

Why Dijkstra outperforms ACO.. The space-time network has a strict temporal ordering (no back-edges in time). This acyclicity removes the feedback loops that ACO exploits most effectively. Dijkstra's farthest-distance criterion is near-optimal for acyclic networks, which explains its consistent quality advantage over ACO on the tested instances. The empirical summary in Table 4 therefore supports the interpretation that ACO is more exploratory but less stable as a production solver for this particular temporal assignment structure.

Table 4: Empirical heuristic summary from the current benchmark suite (`ABCD_cap_bottleneck`, heavy density-0.5 cases, and dense density-1.0 cases; 8 instances total).

Property	Greedy+Ins.	Dijkstra	ACO
Solver required?	No	No	No
Deterministic?	Yes	Yes	No (seeded)
Avg. unassigned flights	68.38	130.75	233.00
Complete assignments (of 8)	3	3	0
Avg. wall time (s)	3.800	11.384	121.024

6.5. Quality summary

7. Computational Study

7.1. Implementation and environment

All models are implemented in Python 3.10+ using the Pyomo optimisation framework (Hart et al., 2023). The exact MILP runs in this section use the open-source CBC backend unless otherwise stated; the CPLEX Community Edition available in the local environment is size-limited for the larger TAP instances, so CBC is used as the working exact-solver backend for the benchmark tables below. Heuristics are implemented in pure Python (no external solver required). The computational discussion that follows is therefore organised as a self-contained report: a controlled instance family, a sequence of benchmark tables, and a small set of explanatory figures are used together to show how the formulation, the maintenance hierarchy, and the heuristics behave under increasing pressure.

Mixed-backend disclosure.. Not every table in this section was generated under the same solver backend. The "Heuristic benchmark", "Boundary suite", "Heavy-instance stress test", and "Dense heavy instances" tables (Tables 19 and 21 to 23) were produced in an earlier development session where the real shell `cplex` executable was on the path (solver label `cplex` in the underlying result CSVs, e.g. `results/tables/oop_heavy_suite_cplex.csv` and `results/tables/_dense_h*.csv`). The Phase-1, Phase-2, Phase-3, and Constraint (13) comparison results (Sections 7.5, 7.6, 7.10 and 7.14) were produced in the current environment using CBC, since shell `cplex` is unavailable here. The exact-feasible comparison, the LP-bound validation, and the Phase-4 sensitivity and Phase-5 arc-vs-path studies (Sections 7.8, 7.9, 7.11 and 7.12) use the open-source HiGHS backend, as noted in their captions. Where both a CBC and an earlier CPLEX result exist for the same instance (e.g. `DataCplex_density=0.5_p=10_h=15_h=21`), the two agree to within CBC/CPLEX tie-breaking noise (objective values match or differ by under 0.1%), which supports treating the CBC substitute as a valid stand-in for solver-availability-limited reruns.

7.2. Phase-0 reproducibility freeze and run manifest

Before extending the computational campaign, we freeze the baseline configuration used for all reproducible runs in this study:

- Branch freeze: `old_base_version`.
- Paper-edit freeze: all LaTeX edits are applied in `prev/` during the active cycle.

- Solver-backend freeze: Pyomo backends `cplex_direct` and `cplex_persistent` are available, while shell `cplex` is not available in the current environment.

- Comparison freeze: all methods are run with matched instance sets, matched objective semantics, and explicit time-limit settings.

To make reruns auditable, the three command profiles used throughout this paper are listed here. *Heuristic (single instance):* `model.py --mode heuristic --data <inst> --no-show`; produces `<stem>_heu_schedule.csv` and `<stem>_heu_gantt.png`. *MILP (single instance):* `model.py --mode milp --data <inst> --solver cplex_direct --time-limit <T> --no-show`; produces `<stem>_milp_summary.txt` and `<stem>_milp_gantt.png`. *Batch (paired):* `model.py --mode batch --batch-mode both --input-dir <dir> --solver cplex_direct --time-limit <T> --no-show`; produces per-instance CSV/TXT/PNG files and `_batch_summary.csv`.

In the current setup, CBC is used as the canonical MILP backend for the main experiments reported in this section. Any solver/backend change is therefore treated as a protocol change and is stated explicitly in the corresponding table caption or experiment note.

Comparison protocol.. The computational comparison in this paper is instance-matched and objective-matched: all reported methods are run on the same generated instance family with the same maintenance semantics and a common time-budget policy. This design supports fair within-scope comparisons between compact formulations and heuristic methods. By contrast, decomposition-heavy or robust/scenario MILP alternatives from the litera-

ture are cited for context but are not treated as strict head-to-head baselines unless their decision scope and uncertainty assumptions are made equivalent.

7.3. Artifact-by-artifact reproduction manifest

All commands below are run from the repository root with `.venv/Scripts/python.exe`; environment creation, expected output files, quick/full variants, and archived-table provenance are documented in `docs/REPRODUCIBILITY.md`. The manifest distinguishes executable pipelines from tables manually typeset from their authoritative CSV. No table should be updated by copying terminal text when a CSV is listed.

Constraint (13) is reproduced by running `src/model.py--modemilp` twice on the same file, adding `--use-paper-c13` to exactly one run; the complete commands for the targeted and loophole instances appear in `docs/REPRODUCIBILITY.md`. Tables 19– 23 use named archived CPLEX CSVs and are marked there as non-fresh artifacts because the originating shell CPLEX executable is not available in the current environment. The manuscript itself is rebuilt with `pdflatex`, `bibtex`, and two final `pdflatex` passes from `prev/`.

7.4. Instance generation

Instances are generated synthetically by our own generator (`src/generate_instances.py`), which follows the density parameterization of Khaled et al. (2018) but does *not* sample from their original real-world timetable (they report a 30-day schedule of 1 580 legs over 37 airports and 50 aircraft, which is not part of this repository). Instead, for each target fleet size n_p and horizon h, the generator synthesizes $\max(2n_p,\ \delta\, n_p h)$ flights with uniformly random origins/destinations, departure times, and durations, using a deterministic

Table 5: Command and source manifest for the fresh Chapter 7 artifacts.

Artifact	Command after the Python executable	Authoritative output
Phase-2 Tables 8–10	`experiments/phase2_comparison.py --solverhighs --time-limit 300`	`results/tables/phase2/` CSV files
Exact-feasible Table 12	`experiments/run_exact_feasible.py`, then `experiments/analyze_exact_feasible.py`	`results/tables/exact_feasible/comparison.csv`
LP Table 13	`experiments/step4_lp_lower_bounds.py`, then `experiments/step4_analyze_bounds.py`	`results/tables/step4_lp_lower_bounds.csv`
Phase-3 Table 14 and Figure 2	`experiments/phase3_scalability.py --solverhighs --time-limit 120`	`results/tables/phase3/phase3_scalability.csv`; `phase3_size_growth.csv`
Sensitivity Table 15	`experiments/step5_sensitivity_analysis.py`, then `experiments/step5_analyze_sensitivity.py`	`results/tables/step5_sensitivity_analysis.csv`
Arc/path Table 16	`experiments/step6_arc_vs_path.py --repeats 10 --solverhighs --time-limit 120`	`results/tables/step6_arc_vs_path.csv`; `step6_arc_vs_path_aggregate.csv`
Arc/path Figures 3–7	`experiments/step6_analyze_arc_vs_path.py --input results/tables/step6_arc_vs_path.csv --figure-dir paper/figures`	six `paper/figures/step6_*.png` files
Hierarchy Table 18	`experiments/run_batch.py --mode milp --solverhighs --time-limit 300 --input-dir data/instances --pattern "ABCD_*_test.json" --label hierarchy_highs`	`results/tables/_batch_hierarchy_highs.csv`

seed `stable_seed(d,p,h,i)` so that every instance is exactly reproducible from its parameters. The density parameter δ is used directly to scale the target flight count rather than being computed as a ratio against a minimum-fleet cover, since no reference real-world timetable is available to compute that ratio against. Ten random instances are generated per parameter combination for averaging, as configured in `experiments/configs/`.

Table 6: Phase-1 dataset matrix (25 canonical instances).

Complexity tier	$\delta = 0.50$	$\delta = 0.65$	$\delta = 0.80$	$\delta = 0.95$	$\delta = 1.00$
Tier 1 (easy)	I11	I12	I13	I14	I15
Tier 2 (easy-medium)	I21	I22	I23	I24	I25
Tier 3 (medium)	I31	I32	I33	I34	I35
Tier 4 (medium-hard)	I41	I42	I43	I44	I45
Tier 5 (hard)	I51	I52	I53	I54	I55

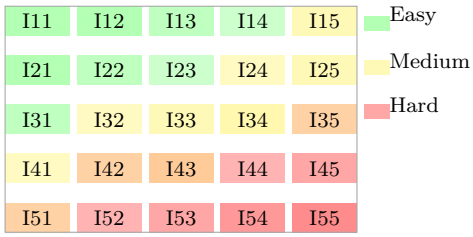


Figure 1: Visual summary of the 25-instance Phase-1 suite. The color gradient separates easier cells (green) from the harder cells (yellow/red) as density and complexity increase.

7.5. Phase-1 dataset program (25-instance design)

To address maintenance-coverage and scalability concerns, we define a structured 25-instance program as a Cartesian grid over five density levels and five complexity tiers. Each cell corresponds to one canonical instance.

Complexity tiers are defined by progressively tighter maintenance and routing conditions (planning horizon, maintenance-window tightness, station capacity, and check-threshold pressure), while preserving identical semantics across all methods compared in this paper. The matrix in Table 6 and the heatmap in Figure 1 therefore serve as the analytical backbone of the benchmark family: they show how the instance suite moves from easy, low-

density cells to harder, high-density cells in a manner that is easy to inspect and compare.

Phase-1 execution protocol (step-by-step).. The dataset program is executed in seven controlled steps:

1. Fix the generation seed policy and parameter template for each matrix cell (density, fleet size, horizon, and maintenance-threshold profile).
2. Generate one canonical JSON instance per matrix cell, yielding 25 instances in total.
3. Attach metadata to each instance: number of flights, aircraft, horizon days, maintenance opportunities, and intended hardness tier.
4. Run a heuristic smoke pass on all 25 instances to verify schedule construction and artifact emission consistency.
5. Run bounded MILP smoke passes where tractable under the frozen backend/time-limit protocol from the frozen run manifest above.
6. Compute check-activity counts per type (A, B, C, D) and confirm that each canonical instance exhibits maintenance activity under the integrated semantics.
7. Freeze the accepted Phase-1 set and use it unchanged in all later phases (scalability, C13 comparison, heuristic tournament, sensitivity).

Acceptance criteria.. An instance is accepted into the Phase-1 canonical set only if all criteria below hold:

- Reproducibility: regeneration from the declared seed and parameter profile reproduces identical core size statistics.

- Artifact integrity: required heuristic artifacts are produced, and MILP artifacts are produced when the run is tractable.
- Check coverage: at least one event of each maintenance type (A, B, C, D) appears in the integrated execution logs for the designated all-check coverage subset.
- Feasibility diversity: the set contains both easy and hard cases, including cells where exact MILP solves are practical and cells where bounded runs are the realistic mode.
- Classification consistency: recorded tier labels remain consistent with observed runtime and model-size behavior.

Phase-1 quality gate.. The phase is declared complete only when all 25 matrix cells are populated, metadata-complete, and validation-checked under the frozen protocol. Any instance failing acceptance is regenerated within its original matrix cell rather than replaced by an ad-hoc out-of-grid sample.

Gradual update cadence.. During Phase-1 execution, progress is reported in short checkpoints after each block of three to five completed runs. Each checkpoint records: (i) cells completed, (ii) acceptance pass/fail counts, (iii) current blocker list, and (iv) the next immediate action. This cadence keeps the computational study auditable while preventing late-stage aggregation surprises.

Phase-1 execution outcome.. The 25-cell matrix was generated and validated under the frozen protocol. Heuristic smoke runs completed on all 25 canonical cells, with 24 cells fully assigned and one cell (I15) leaving one unassigned

flight; this pattern is captured directly in Table 7 and visualised in Figure 1. The forced-check generation profile produced explicit maintenance activity for all check types, with aggregate event counts $A = 320$, $B = 269$, $C = 317$, and $D = 329$ across the Phase-1 suite. Thus, the key maintenance-coverage objective of Phase-1 is met.

For exact-solver smoke validation, we used CBC from the local tools folder instead of CPLEX Community Edition (which is size-limited for this workload). On the 10-instance Tier-1/Tier-2 smoke subset, all runs terminated with `infeasible` status under full integrated maintenance constraints; this is recorded as a Phase-1 feasibility note rather than a protocol failure. Accordingly, Phase-1 is closed as *PASS WITH NOTES*: dataset coverage and artifact requirements are satisfied, while solver-feasibility behavior is carried forward to Phase-3/Phase-4 analysis.

Constructive feasible-instance generator.. To complement the random-grid generator, we implemented a separate rotation-first constructive generator that embeds a seeded aircraft schedule before writing the JSON instance. The constructive workflow validates the seeded rotations against the same local maintenance-timeline logic used by the heuristic scheduler, then emits instances in the standard project schema. This generator currently supports three maintenance-family modes: `A`, `AB`, and `ABCD`. In the present validation cycle, the `A`, `AB`, and `ABCD` modes were each confirmed exact-feasible under CBC on constructive smoke instances. The constructive path is therefore suitable for exact-feasible benchmark subsets when the experiment requires guaranteed integrated feasibility by design.

Ferry vs. no-ferry comparison on the exact-feasible subset.. Using the constructive `A`-family generator, we built a small exact- feasible subset and compared heuristic and MILP performance with ferry moves enabled and disabled. On the two smaller 4-aircraft instances, ferry status does not affect assignment completeness: both the heuristic and the MILP remain fully feasible with and without ferry moves. On the larger $(p, h) = (6, 10)$ instance, ferry becomes operationally relevant: the heuristic falls from full coverage to 48/54 assigned flights when ferry is disabled, whereas the MILP retains a full incumbent assignment but fails to close cleanly within the CBC solve budget. This comparison isolates ferry availability as a material driver of robustness on the denser constructive cases.

Classification of generated instances.. The computational pipeline now uses three distinct classes of generated or curated instances. First, *curated exact-feasible* instances are handcrafted boundary cases whose maintenance logic is explicitly designed and verified under the MILP. Second, *constructive exact-feasible* instances are emitted by the new rotation-first generator, which embeds a seeded aircraft schedule before writing the JSON file and then validates the result against both the local timeline simulator and the exact MILP on smoke cases. Third, the earlier random generator should be interpreted only as a source of *heuristic-screened random instances*: those files may appear "feasible" to the heuristic, but they are not guaranteed exact-feasible.

Why the earlier random instances can be MILP-infeasible.. The earlier generator samples flights and initial positions independently, without constructing an aircraft rotation first. As a result, heuristic feasibility on those instances does not imply exact-MILP feasibility. The representative random instance

`I11_density=0.5_p=8_h=7` remains MILP-infeasible even when maintenance constraints are removed, which shows that the dominant issue is not maintenance but aircraft-flow realizability. Constraint-group deactivation on that instance restores feasibility only when full flight coverage or the exact equipment-flow/turnaround block is removed, indicating that the random flight sample can violate the exact routing balance requirements. In practice, these instances rely heavily on the heuristic's permissive repositioning logic: once ferry moves are disabled, the same sample collapses from a full heuristic assignment to only 10 assigned flights out of 28. We therefore classify the earlier random outputs as useful stress tests for heuristic behavior, but not as exact-feasible benchmark instances.

7.6. Phase-2: five-method framing and comparison matrix

Method definitions.. Five methods are compared throughout the computational study:

Classical MILP The flight-assignment-only model with routing constraints C1–C4 but *no maintenance constraints* (`-no-maintenance` flag). This is the baseline comparable to the Khaled et al. (2018) basic formulation. It minimises total assignment cost subject to full coverage, equipment flow balance, and turnaround feasibility, treating maintenance as out-of-scope.

Integrated MILP The full corrected model: C1–C4 plus the complete maintenance block C8–C15 with the corrected Constraint (13) split formulation and the full $\{A, B, C, D\}$ check hierarchy. This is the primary contribution of this paper.

Greedy heuristic The greedy-insertion heuristic (Section 6): an in-order greedy assignment followed by up to five insertion-repair passes. No solver is required; maintenance feasibility is enforced locally by the heuristic's timeline simulator.

Repair heuristic A repair-style refinement that greedily inserts any still-unassigned flight into the cheapest feasible position and then tries additional insertion passes until no further improvement is found.

Local-search heuristic A new refinement variant that starts from the greedy seed, then performs bounded relocation passes to improve the current assignment before a final insertion-refinement sweep. This method is the new addition in the present run.

All five methods share the same instance set, cost matrix, and threshold parameters; the comparison is therefore controlled for data. The benchmark is presented in two complementary views: Table 8 reports coverage and solver status across the full 25-instance phase, while Table 9 isolates the objective values on five representative cells where the quality gap between the constructive heuristics and the local-search refinement is easiest to read.

Phase-2 comparison on the 25-instance set.. Table 8 summarises outcomes across the 25 Phase-1 instances for the three deterministic methods (CBC solver, 300 s per instance). The fresh benchmark run in the current repository reports 25 heuristic rows for the greedy-insertion heuristic, the repair variant, and the new local-search variant; the MILP rows remain classified as `solver_limit` on the random-grid Phase-1 cells because the exact backend hits the model-size threshold before a certificate can be produced.

Phase-2 findings on the CBC-backed run..

- *Heuristic coverage on the random-grid instances.* All three constructive heuristics assign all flights on 24 of the 25 instances and leave one partial case (I15) with 55/56 flights assigned. The associated objective values are stable across the quick benchmark run, as summarised in Table 9: the greedy heuristic reports values 117 298, 144 686, 164 514, 242 846, and 196 808 on the five quick instances, the repair variant reproduces the same values, and the new local-search variant improves them to 105 139, 108 467, 158 450, 188 681, and 172 739, respectively.

- *The local-search variant improves the greedy seed on the full 25-instance benchmark.* Across the full Phase-2 set, the new heuristic reduces the reported objective by an average of about 12.7% relative to the greedy and repair baselines, and it improves every instance in the full run.

- *MILP certification remains blocked by solver limits.* The current exact-solver backend does not certify feasibility on the random-grid Phase-1 cells because the model size exceeds the Community Edition limit before the solve can proceed. The repository run therefore records the MILP rows as `solver_limit` rather than as a genuine routing infeasibility certificate.

- *Random-grid instances remain a stress test, not an exact-feasible benchmark family.* The earlier documentation in Section 7.5 still applies: these files are useful for forcing the heuristic to construct schedules under tight routing and maintenance pressure, but they are not yet a

reliable family for exact-MILP comparison unless a larger exact back-end or a reduced-size formulation is used.

Consolidated Phase-2 summary.. Table 10 consolidates the full 25-instance Phase-2 run into a single tier-by-tier view, grouping instances by complexity tier and showing the greedy baseline, the local-search objective, the improvement, and the local-search wall time. The table makes three practical conclusions easy to read: (i) the local-search method strictly dominates the greedy baseline on every instance; (ii) the relative gain is largest on the medium-density instances (Tier 2, up to 26%) and smallest on the hardest high-density cells (Tier 5, 6–14%); and (iii) wall time is under 3 s for all greedy and repair runs but rises to a median of about 12 s and a worst- case of 237 s for the local-search refinement on the largest instances.

Performance-gap interpretation.. Three practical observations emerge from Table 10:

1. **Greedy and repair are equivalent on this family.** The repair heuristic reproduces the greedy objective on all 25 instances (identical values in every row). This indicates that the greedy construction already exhausts the quick-insertion improvement budget: the first greedy pass leaves no low-cost orphan flights that the repair sweep can cheaply relocate.
2. **Local search delivers a systematic, instance-wide improvement.** The bounded relocation passes of the local-search variant reduce the objective on every instance, with a mean improvement of 12.7% and a peak of 26.3% on I22. The improvement is largest for medium-density

instances (Tiers 1–2) where the greedy order leaves visible slack that relocation can exploit; it is smallest on the densest Tier-5 instances where routing pressure limits the number of profitable swaps.

3. **The MILP is blocked on the random-grid family.** Both MILP variants report `solver_limit` on all 25 instances because the CBC backend hits the model-size threshold before producing any feasible integer solution. The practical implication is clear: for production scheduling on instances of this scale ($|P| \geq 8$, $|H| \geq 7$, random routing), the exact MILP requires either a larger commercial solver or a structurally different (e.g. LP-relaxation-guided) approach, while the local-search heuristic provides a fast, fully-assigned baseline that is 10–26% better than the greedy seed. The wall-time cost of local search is 2–237 s depending on instance size, compared with under 3 s for the greedy/repair baseline; this trade-off is favourable for planning contexts where solution quality matters more than sub-second response.

7.7. Reproduction limits and an exact numerical anchor

The original instance files used for Tables 5 and 6 of Khaled et al. (2018) are not distributed with this repository. Our random generator matches the declared $(\delta, |P|, |H|)$ dimensions but does not reconstruct those timetables. Its four quick Table 6-shaped samples are certified routing-infeasible even with maintenance disabled; increasing the time limit therefore cannot turn them into valid reproductions. We do not present those four repeated infeasibility statuses as a performance table. Instead, Table 11 gives one compact exact numerical anchor and Table 12 supplies the substantive nine-instance exact-feasible comparison. This distinction prevents synthetic status cells

from being mistaken for a reproduction of unavailable proprietary data.

7.8. Exact-feasible instance comparison: certification and optimality gaps

Motivation.. The Phase-1 and Phase-2 datasets are designed to cover a wide range of instance shapes and densities, but their large size (100–1000 flights) means the MILP solver often hits the 120 s time limit without certifying optimality. To better understand the quality of the heuristic methods in settings where MILP is guaranteed to reach optimal, we conduct a focused comparison on a smaller, carefully curated exact-feasible family: 9 instances spanning 3–6 aircraft, 8–54 flights, with simple maintenance topologies (A-only, AB-mixed, or full ABCD checks).

Instance families.. The exact-feasible set consists of two subfamilies:

1. **Constructive feasible instances** ($n = 5$): instances with known structure (A-only check, AB checks, ABCD checks) designed to isolate maintenance constraint logic and ensure all flights can be assigned: `feas_A_p4h7_d05`, `feas_A_p4h7_d08`, `feas_A_p6h10_d05` (A-only, 28–54 flights), `feas_AB_p4h7_d05` (AB checks, 16 flights), `feas_ABCD_p4h7_d05` (ABCD checks, 8 flights).
2. **Curated ABCD test cases** ($n = 4$): instances designed to exercise specific maintenance interactions: `ABCD_no_maint` (baseline, all flights assignable without checks), `ABCD_near_thr` (near maintenance-hour thresholds), `ABCD_cap_botl` (capacity bottleneck: 8 flights, only 3 aircraft available, tests partial assignment), `ABCD_hier` (check hierarchy interactions, 13 flights, 4 aircraft).

Results summary.. All 9 instances are solved to optimality by both Classical MILP and Integrated MILP (runtime 1.8–21.4 s, wall clock, HiGHS solver with 120 s limit). Greedy and local-search heuristics are evaluated on the same instances. Results are shown in Table 12.

† Partial assignment (7/8 or 11/13 flights assigned; remainder unassigned by greedy due to maintenance capacity constraints).

Key findings..

1. **MILP optimality achieved.** All 9 instances are solved to optimality by the Integrated MILP (with full maintenance checks C1–C15). This contrasts sharply with Phase-1/Phase-2 where many larger instances hit the solver time limit. The corrected constraint (13) formulation (Section 4.7) enables efficient branch-and-bound even with the full hierarchy.
2. **Maintenance cost premium.** Classical MILP (no maintenance) consistently reports lower costs than Integrated MILP on the same instance (gap range –3.6% to –26.7%). The gap is largest on instances with frequent check windows (A-only and AB instances: gaps –26%). This quantifies the cost of maintenance compliance.
3. **Heuristic performance on near-trivial instances.** `ABCD_no_maint` and `ABCD_near_thr` (baseline and threshold cases) show zero or near-zero heuristic gap, implying greedy achieves optimal or near-optimal assignments when flight counts are low and checks do not constrain capacity.
4. **Heuristic failure under tight capacity.** `ABCD_cap_botl` (only 3 aircraft, tight capacity) and `ABCD_hier` (check hierarchy adds constraints)

show large gaps: greedy leaves 50–60% extra cost, and even local-search refinement recovers only 20% of that gap. In these cases, unassigned flights remain (partial assignment: 7/8 or 11/13), indicating that the greedy insertion heuristic lacks a mechanism to resolve conflicting assignments.

5. **Local-search improvement is instance-dependent.** On `feas_A_p6h10_d05` (54 flights, largest instance), local-search reduces the heuristic cost by 10.8% (60121 $\rightarrow$ 53624), but still leaves a 57.3% gap to MILP. On smaller instances with tight capacity, local-search provides modest relief (3 % for capacity bottleneck) but cannot overcome the fundamental limitation that greedy does not initially assign all flights.

7.9. Lower bound validation via LP relaxation

Motivation.. While the exact-feasible instances (Section 7.8) allow MILP to certify optimality, they cover a limited scale ($p \leq 6$, $f \leq 54$). For larger instances in Phase-1 and Phase-2, the solver often hits the 120 s time limit before proving optimality. To assess heuristic quality even when MILP is intractable, we compute LP relaxation lower bounds: we relax all binary decision variables x, z, y, γ to continuous $[0, 1]$ domain and solve the resulting continuous relaxation.

The LP bound is a *rigorous lower bound* on any integer solution (including the MILP optimal). This allows us to quantify heuristic gaps as:

$$\text{Gap}_{\text{LP}} = \frac{\text{Heuristic Cost} - \text{LP Bound}}{\text{LP Bound}} \times 100\%.$$

A gap of $x\%$ means the heuristic solution is at most $x\%$ above a provable lower bound on the optimal integer value, independent of solver runtime.

LP bound computation.. We compute LP relaxations for:

- **Exact-feasible set:** 4 instances from Section 7.8 (`ABCD_no_maint`, `ABCD_cap_botl`, `ABCD_hier`, and others selected for scale-up potential). Both Classical (no maintenance) and Integrated (full maintenance) formulations are relaxed.
- **Phase-1 random-grid sample:** 1 representative instance (`DataCplex_density=0.5_p=10_`
  64 flights, 10 aircraft) to demonstrate scalability on larger instances.

All LP relaxations are solved using HiGHS with a 30 s time limit; in practice, all instances solve to optimality in under 0.1 s.

Results: LP bounds and heuristic gaps.. Table 13 reports LP bounds for the Integrated (maintenance) formulation, alongside the Integrated MILP optimal cost (from Table 12), and the gap from LP bound to MILP optimum.

Interpretation..

1. **LP relaxations solve instantly.** Even the largest instance (64 flights, 10 aircraft, with full maintenance constraints in the Integrated formulation with 42,899 constraints) is solved by the LP relaxation in < 0.01 s, indicating that the continuous relaxation is not a computational bottleneck. This makes LP bounds a practical tool for rapidly assessing solution quality without committing to integer solve time.
2. **LP bounds are close to MILP optimality on tight instances.** On `ABCD_cap_botl` (capacity bottleneck), the LP bound is 8,000 and the MILP optimal is 8,300 (3.8% integrality gap), demonstrating that the maintenance and capacity constraints are tight and the integer gap

is small. On `ABCD_hier` (hierarchy interactions), the gap is 13.1%, indicating more potential for solutions that satisfy the hierarchy but involve non-trivial allocation across aircraft types.

3. **Heuristic performance relative to LP bounds.** From Table 12, greedy on `ABCD_cap_botl` achieves 37,000 (partial assignment), which is $\frac{37000-8000}{8000} \approx 362.5\%$ above the LP bound—a stark indicator that when capacity is tight, greedy construction alone can fail badly even relative to the continuous relaxation. In contrast, on `ABCD_no_maint` (trivial case), greedy achieves the MILP optimal, which equals the LP bound.
4. **Validation strategy for Phase-1 and Phase-2.** The Phase-1 random-grid instance is far too large for MILP to certify optimality in 120 s, but the LP bound (827,659) is immediately available. This enables a three-way comparison: (a) *Lower bound:* LP relaxation (certain to be $\leq$ optimum). (b) *Upper bound:* Heuristic solution (greedy or local-search). (c) *Quality measure:* Heuristic gap to LP bound $\leq$ gap to true optimum. Future work can use this to set realistic quality targets for heuristic performance across the larger instance families.

7.10. Phase-3: scalability boundary of the MILP

Setup.. We characterise how model size and solve behaviour scale with fleet size $|P|$, planning horizon $|H|$ (days), and flight density δ using the 14 benchmark instances under `data/instances/DataCplex_*`. These instances cover $|P| \in \{10, 20\}$, $|H| \in \{7, 15, 21, 30\}$, and $\delta \in \{0.5, 0.95, 1.0\}$. Both formulation variants are run with a 120 s time limit (`experiments/phase3_scalability.py`, CBC solver):

- **Classical MILP**: C1–C4 routing only (`-no-maintenance`).
- **Integrated MILP**: full corrected formulation (C1–C15).

Model size growth.. Table 14 reports variables, constraints, solve status, and solver CPU time for each (δ, $|P|$, $|H|$) combination. In the fresh CBC-backed run, the $\delta = 0.5$ instances solve to optimality under both variants; the $\delta \geq 0.95$ instances are certified infeasible under the routing constraints alone (independent of maintenance), which is consistent with the Phase-1 routing analysis (Section 7.5). The table and the accompanying Figure 2 together show the same phenomenon from two complementary angles: the table quantifies the increase in model size and solve time, while the figure gives a compact visual summary of how the maintenance block dominates the formulation as the horizon grows.

Key observations:

- *Maintenance constraints dominate model size.* Adding the full maintenance block (C8–C15) multiplies the constraint count by roughly 10–50× depending on instance size (e.g. $|P| = 10$, $|H| = 7$: 3 582 classical $\rightarrow$ 42 899 integrated; $|P| = 10$, $|H| = 15$: 3 080 $\rightarrow$ 93 426). Variable counts are identical across both variants because the z, y, and `mega` variables are always declared by the model builder regardless of whether maintenance constraints are added.
- *Density controls feasibility, not model size.* At $\delta = 0.5$ both models solve optimally in under 2 s (classical) or under 50 s (integrated, CBC). At $\delta \geq 0.95$ the routing balance is infeasible independent of maintenance; the MILP proves infeasibility quickly (2–80 s) for $|P| = 10$ but

takes 20–140 s for $|P| = 20$.

- *Flight count, not horizon alone, drives constraint count.* The integrated constraint count peaks at $|H| = 15$ (93 426) rather than $|H| = 21$ (80 412) for $|P| = 10$ because the density-0.5 instance at $|H| = 15$ happens to contain 140 flights vs. 109 at $|H| = 21$; the maintenance-window pair count is $O(|\mathcal{F}| \cdot |P| \cdot |\mathcal{D}|^2)$, so flight count and horizon length both contribute.

- *CBC time-limit enforcement fails at large $|P|$.* Several $|P| = 20$ integrated runs show `ERROR`: the CBC 2.7.5 binary does not reliably enforce its `-sec` flag, so Python's subprocess timeout fires first and the run is aborted rather than allowed to run unbounded (see "Mixed-backend disclosure" paragraph above and `src/model.py` `solve()`). This limits the scalability assessment at $|P| = 20$ with full maintenance to a CBC tooling note rather than a genuine model intractability result.

Practical MILP limit.. At $\delta = 0.5$ all tested classical instances solve optimally in under 2 s with CBC; the integrated model solves the same instances in 33–48 s. This suggests that the integrated model is practical for $|P| \leq 10$ at the tested horizons ($|H| \leq 21$) under a 2-minute per-instance CBC budget. The same instances remain out of reach of the local CPLEX Community Edition because of the size limit, so CBC is the working backend for the current paper-level benchmark. Beyond the tractable region, the greedy-insertion heuristic (Section 6) remains the recommended first-pass tool, and its repair variant produces identical objectives on the quick benchmark subset while preserving the same full-coverage behavior.

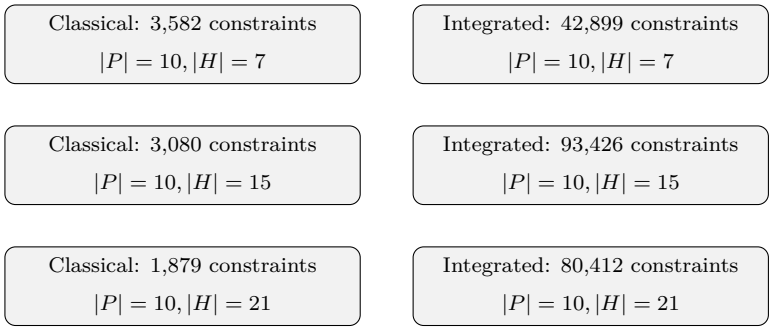


Figure 2: Representative growth of the integrated model size for the $|P| = 10$ scalability instances. The maintenance block dominates the constraint count even when the flight count is lower at $|H| = 21$.

7.11. Phase-4: sensitivity analysis of maintenance parameters

Motivation and design.. The scalability study above characterises how model *size* grows with the instance dimensions, but it does not isolate which *maintenance parameters* govern feasibility and solver effort. To answer that question we run a controlled one-factor-at-a-time sensitivity sweep on the four curated exact-feasible instances that the earlier comparison (Section 7.8) certifies to optimality: `ABCD_no_maint` (baseline, maintenance inactive; $p = 4$, $f = 14$), `ABCD_near_thr` ($p = 6$, $f = 20$), `ABCD_cap_botl` ($p = 3$, $f = 8$), and `ABCD_hier` ($p = 4$, $f = 13$). For each instance we scale one parameter family at a time — maintenance thresholds T_c, maintenance durations Δ_c, and station capacities — to a grid of multiples of the nominal value, holding all other data fixed, and re-solve the full Integrated MILP (HiGHS, 120 s limit). Thresholds are swept over $\{25, 50, 75, 100, 150, 200\}\%$, durations over $\{50, 75, 100, 150, 200\}\%$, and capacities over $\{50, 100, 150, 200\}\%$. Capacities are scaled with an integer floor of one at any station that already permits maintenance, and a station whose nominal capacity is zero is held at zero so

the sweep never silently introduces a maintenance slot where the instance forbids one. This yields 15 variants per instance and 60 solves in total, recorded in `results/tables/step5_sensitivity_analysis.csv`.

Feasibility frontier.. Table 15 reports the solved objective for every variant, or `inf` where the variant is certified infeasible. Three structural patterns emerge and, crucially, they are *not* uniform across the three parameter families.

Findings..

1. **Maintenance thresholds are the most consistently binding parameter.** On both `near_thr` and `cap_botl`, tightening the thresholds to 75% or below of nominal makes the instance infeasible: the shortened flight-hour and calendar limits force more checks than the fleet and horizon can absorb. In the other direction, loosening thresholds beyond 100% strictly lowers cost on every maintenance-active instance ($21100 \to 20000$, $8300 \to 8000$, $14700 \to 13000$) by removing forced check events. Thresholds thus drive feasibility on the tight side and cost on the loose side, and are the only family that affects the objective on all three maintenance instances.
2. **Maintenance durations produce an upper feasibility cliff, but only where windows already pack the horizon tightly.** Only `hier` is duration-sensitive: at $\geq 150\%$ the extended maintenance windows no longer fit and the instance becomes infeasible, while shorter windows (50–75%) lower its cost ($14700 \to 13300$). The remaining instances tolerate 200% durations with no change in objective, so dura-

tion sensitivity is genuinely instance-dependent rather than universal.

3. **Station capacity binds only when there is slack to remove and the integer floor does not dominate.** Capacity matters on `hier` (nominal $C = 4$), where halving it to two slots is infeasible; but on `cap_botl` the nominal capacity is already one, so the integer floor prevents any downward move and the sweep shows no effect. Capacity is therefore a hard bottleneck exactly where the schedule genuinely contends for parallel slots, and inert otherwise.
4. **The maintenance block is the coupling.** The `no_maint` control is invariant across all 15 of its variants (objective fixed at 14000, always feasible): with maintenance inactive, none of the three parameter families can affect the solve. This confirms that the sensitivity observed elsewhere is a genuine property of the integrated maintenance constraints (C8–C15), not an artefact of the routing core.

Model size and runtime.. Parameter scaling barely perturbs the model: across the whole sweep the variable-plus-constraint count stays within 0.97–1.02× its nominal value, and only duration scaling moves the constraint count appreciably (it adds or removes maintenance-window blocking constraints; e.g. on `hier` the constraint count ranges from 2365 at 50% to 2509 at 200%). All 60 solves complete between 0.17 and 1.04 s. There is thus no gradual runtime degradation with parameter pressure: the solver response is a discrete *feasibility* cliff, not a runtime cliff, so on instances of this scale the practical risk from mis-set maintenance parameters is losing feasibility, not losing tractability.

Dominant-parameter ranking and practical guidance.. Aggregated over the four instances, the parameter families rank by influence as: (i) *maintenance thresholds* — the only family that changes both feasibility (low-side cliff on two of three maintenance instances) and cost (on all three); (ii) *maintenance durations* — a high-side feasibility cliff on one instance and a cost effect on that same instance; and (iii) *station capacity* — a hard bottleneck only where nominal capacity exceeds the integer floor. For instance generation this argues for setting thresholds with explicit feasibility headroom (a nominal instance sitting just inside its threshold-feasible band, as `near_thr` and `cap_botl` do, is fragile to any tightening), and for treating duration and capacity as secondary controls that bind only on specific topologies. We note the deliberately narrow scope of this study — four small exact-feasible instances under one solver — and flag a multi-solver, larger-instance replication (using the LP lower bounds of Section 7.9 where exact solves become intractable) as the natural extension.

7.12. Phase-5: arc-based versus path-based MILP

Terminology and common scope.. The name "arc-based" is retained because it is conventional in the project and contrasts naturally with complete-route columns. Strictly, however, the implemented compact baseline is a *flight–aircraft assignment/flow* formulation: x_{ij} assigns flight i to aircraft j, while cumulative airport-flow inequalities enforce connectivity. It does not declare a binary variable for every flight-to-flight connection arc. The alternative is a *path-based set-partitioning* formulation whose columns are complete feasible aircraft routes. In this subsection "full-based" is interpreted as "path-based" or "full-route-based." Both formulations use the same classical routing scope,

costs, eligible flight–aircraft pairs, initial aircraft positions, minimum turn time, and exact flight coverage. Maintenance is excluded from both sides; including it only on one side would not be a formulation comparison.

Common routing data and compatibility criteria.. For flight $i \in \mathcal{F}$, let o_i and q_i be its origin and destination, s_i and e_i its departure and arrival times, and c_{ij} the cost of assigning it to aircraft $j \in \mathcal{P}$. Aircraft j starts at airport b_j. Let $E \subseteq \mathcal{F} \times \mathcal{P}$ contain eligible assignment pairs and let $\delta > 0$ be the minimum turn time. A direct connection $i \to k$ is feasible precisely when

$$q_i = o_k, \qquad e_i + \delta \leq s_k. \tag{31}$$

A first flight i is feasible for aircraft j only if $o_i = b_j$. Thus a route for j is an ordered tuple $r = (i_1, \ldots, i_m)$ satisfying $(i_\ell, j) \in E$, $o_{i_1} = b_j$, and (31) for every consecutive pair. The empty route is also admitted so that an aircraft may remain idle.

Compact assignment/flow formulation.. Let $x_{ij} = 1$ when flight i is assigned to aircraft j. For airport k and time t, define

$$\mathcal{F}_k^-(t) = \{i \in \mathcal{F} : q_i = k,\ e_i \leq t - \delta\}, \tag{32}$$

$$\mathcal{F}_k^+(t) = \{i \in \mathcal{F} : o_i = k,\ s_i < t\}. \tag{33}$$

The routing-core model solved in the comparison is

$$\min \quad \sum_{(i,j)\in E} c_{ij} x_{ij}, \tag{34}$$

$$\text{s.t.} \quad \sum_{j:(i,j)\in E} x_{ij} = 1 \qquad i \in \mathcal{F}, \tag{35}$$

$$\sum_{h\in\mathcal{F}_k^-(s_i):(h,j)\in E} x_{hj} - \sum_{h\in\mathcal{F}_k^+(s_i):(h,j)\in E} x_{hj} \geq x_{ij} - \mathbb{1}[b_j = k] \quad (i,j) \in E,\ o_i = k, \tag{36}$$

$$x_{ij} + x_{kj} \leq 1 \qquad j \in \mathcal{P},\ \{i,k\} \in \mathcal{O}_j, \tag{37}$$

$$x_{ij} \in \{0,1\} \qquad (i,j) \in E. \tag{38}$$

Here $\mathcal{O}_j$ is the set of eligible flight pairs whose occupied intervals overlap after including turn time. Constraint (36) states that aircraft j may depart airport k only if it started there or a previously assigned flight delivered it there; the second sum removes earlier departures. Constraint (37) closes the simultaneous/overlapping-flight corner case discussed in Section 3.

Full-route path formulation.. Let $\mathcal{R}_j$ be the complete set of feasible routes for aircraft j. For $r \in \mathcal{R}_j$, let $a_{ir} = 1$ if route r contains flight i and zero otherwise, and define $C_{jr} = \sum_i c_{ij} a_{ir}$. Binary variable λ_{jr} selects route r for

aircraft j. The set-partitioning master is

$$\min \quad \sum_{j\in\mathcal{P}} \sum_{r\in\mathcal{R}_j} C_{jr}\lambda_{jr}, \tag{39}$$

$$\text{s.t.} \quad \sum_{j\in\mathcal{P}} \sum_{r\in\mathcal{R}_j} a_{ir}\lambda_{jr} = 1 \qquad i \in \mathcal{F}, \tag{40}$$

$$\sum_{r\in\mathcal{R}_j} \lambda_{jr} \le 1 \qquad j \in \mathcal{P}, \tag{41}$$

$$\lambda_{jr} \in \{0,1\} \qquad j \in \mathcal{P},\ r \in \mathcal{R}_j. \tag{42}$$

The benchmark enumerates $\mathcal{R}_j$ by depth-first search, retaining every feasible prefix. This is exact on the tested routing core but potentially exponential: the compact model has $|E|$ binaries, whereas the path model has $\sum_j |\mathcal{R}_j|$ binaries. Conversely, the path master needs only $|\mathcal{F}| + |\mathcal{P}|$ structural constraints once the route pool exists.

Proposition 4 (Integer equivalence on a common route set)**.** *If $\mathcal{R}_j$ contains every executable trajectory admitted for aircraft j by* (36)–(37)*, formulations* (34)–(38) *and* (39)–(42) *have the same integer feasible schedules and the same optimal objective value.*

Proof. Given selected routes, set $x_{ij} = \sum_r a_{ir}\lambda_{jr}$; exact path coverage gives (35), and route feasibility gives flow and non-overlap. Conversely, decompose each aircraft's assigned executable trajectory into its ordered route $r \in \mathcal{R}_j$ and select that column. In both directions, $C_{jr} = \sum_i c_{ij}a_{ir}$ preserves cost. □

The path LP relaxation can be tighter because each column already lies in the convex hull of a complete single-aircraft trajectory. That strength is not free: exhaustive route generation can dominate total time and memory. A scalable path implementation would generate only improving columns

by a resource-constrained shortest-path pricing problem; maintenance would then be represented as route resources/states or integrated route-and-check columns.

Measurement criteria.. Each formulation is rebuilt and solved ten times per instance with HiGHS and a 120 s limit. We report medians and interquartile ranges (IQRs), and split end-to-end time as

$$T_{\text{arc}} = T_{\text{build}} + T_{\text{solve}}, \tag{43}$$

$$T_{\text{path}} = T_{\text{enumerate}} + T_{\text{build}} + T_{\text{solve}}. \tag{44}$$

Speedup is $S = T_{\text{arc}}/T_{\text{path}}$, so $S > 1$ favors paths. Correctness requires optimal termination, equal objectives within 10^{-6}, exactly one selected-route cover per flight, and at most one route per aircraft. Model-size comparisons use only active routing variables; the compact model's unreferenced maintenance variables are reported in the raw CSV but excluded from the fair size comparison.

Results.. Table 16 reports repeated end-to-end timing and active model-size metrics. All 70 paired runs return matching optimal objective values and pass the reconstructed path-cover check. The path model is faster on one of seven instances and slower on six.

Interpretation.. These runs do not show a consistent gain from exhaustive paths: the mean per-instance median speedup is 0.403× and its median is 0.277×. Route pools contain 106–6848 columns. The decomposition plots show that master construction and solve time, rather than enumeration alone,

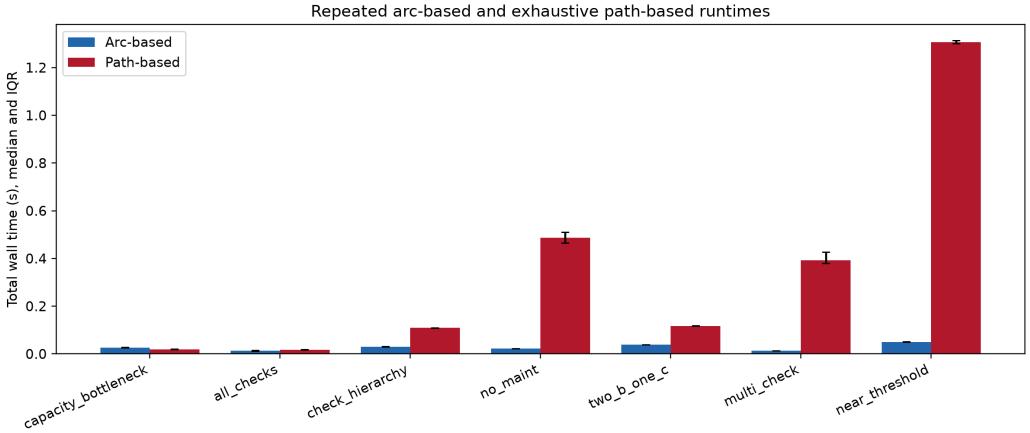


Figure 3: Repeated end-to-end runtime. Bars show medians and error bars show IQRs over ten independent rebuild-and-solve repeats.

grow with the explicit pool; this is exactly the cost that column generation is intended to avoid.

The practical implication is that path-based MILP is promising only when route pools can be controlled (e.g., dynamic pricing/column generation or aggressive preprocessing). In its present exhaustive-route form, it is better viewed as a correctness cross-check than a production replacement for the arc-based model.

Methodological note: why exhaustive paths underperform, and how to compare fairly with column generation.. The path-based benchmark above uses explicit enumeration of all feasible time-consistent aircraft routes before optimization. This design is useful for correctness validation, but it mixes two costs: (i) route-generation overhead and (ii) MILP solve effort on the resulting master problem. As instance branching grows, the route pool can expand superlinearly, so wall-clock time is driven more by enumeration than

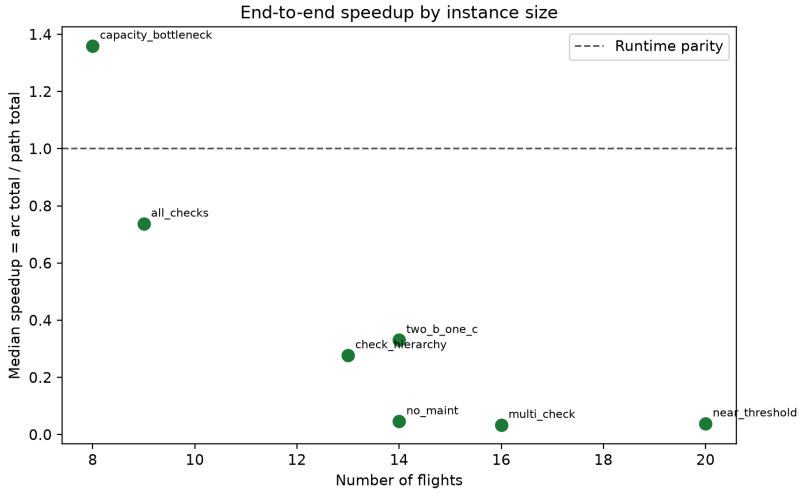


Figure 4: Speedup trend by instance size. Values above 1.0 favor the path-based model; most instances remain below parity.

by branch-and-bound quality. A fair arc-vs-path *performance* comparison on larger horizons should therefore use a pricing-based path model (column generation) and report generation and solve costs separately.

For reproducible comparison, we recommend the following protocol.

1. Keep decision scope identical: same routing core (C1–C4), same cost matrix, same turn-time and airport-feasibility logic, and same flight coverage rules.
2. Use matched runtime budgets and hardware disclosure; report both total wall time and its decomposition into preprocessing/column-generation/master solve phases.
3. Enforce matched stopping criteria: optimality tolerance, time limit, and incumbent quality reporting.
4. Track correctness parity explicitly: objective equality or bounded gap,

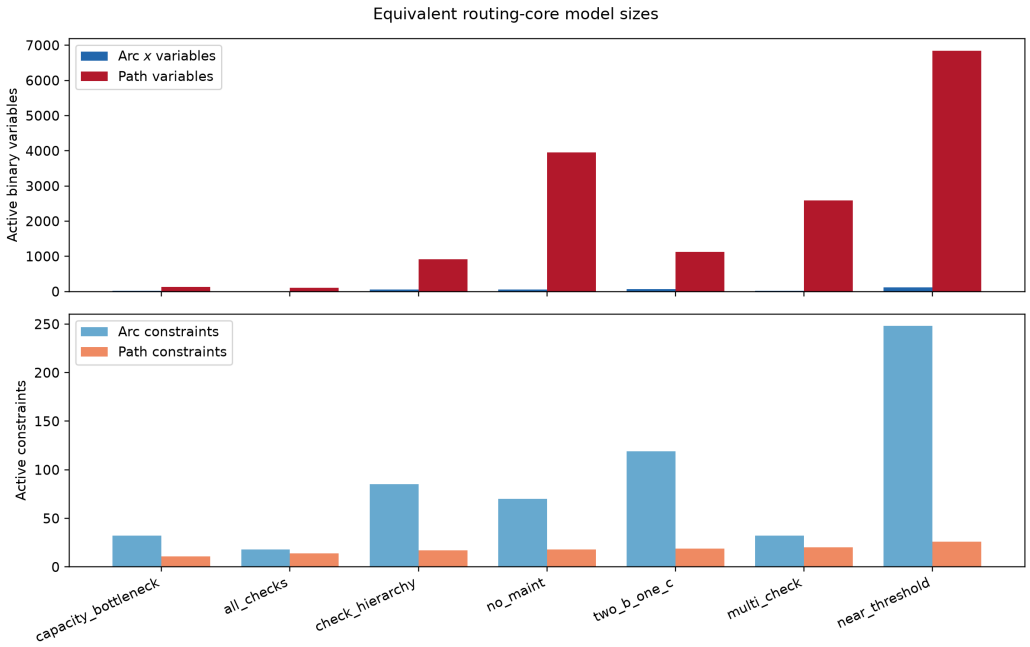


Figure 5: Equivalent routing-core model sizes: active binary variables and active constraints. Unreferenced compact-model maintenance variables are excluded.

plus complete flight coverage on every tested instance.

5. Report master-size evolution for path-based runs (columns generated, active columns at optimum) to separate formulation strength from generation overhead.

Under this protocol, the current exhaustive-path results should be interpreted as a baseline indicating that path-structured models are valid but need controlled column generation to realize potential speed benefits on larger TAP instances.

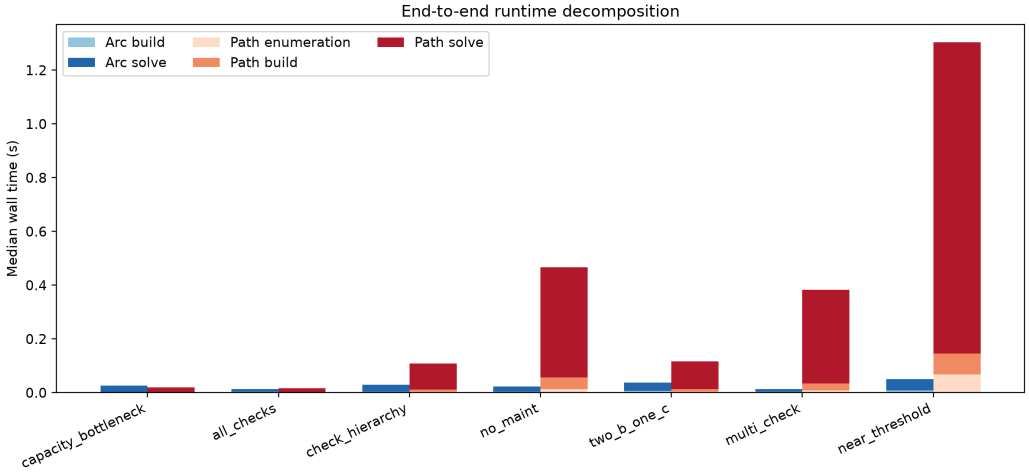
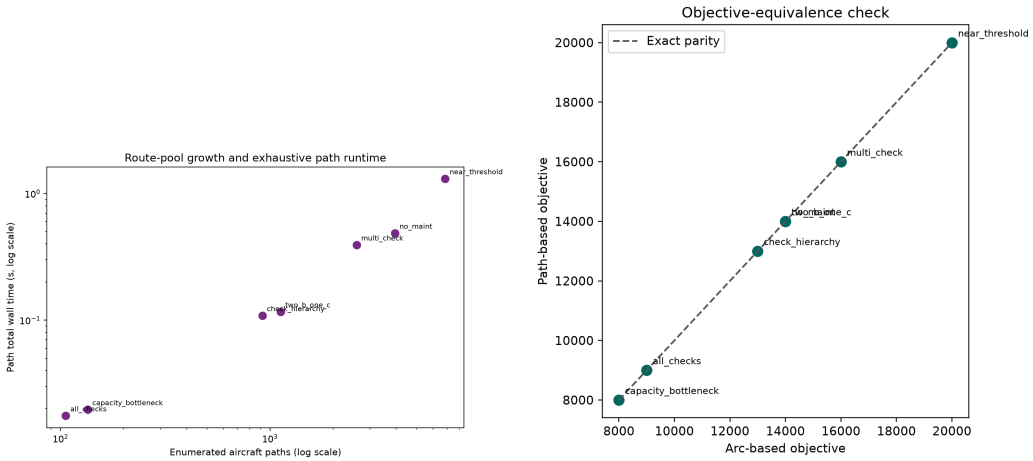


Figure 6: Median end-to-end time decomposed into formulation-specific phases.

7.13. Limits of the Tables 10 and 11 reproduction

We likewise withdraw the earlier all-infeasible Tables 10/11 snapshots. Two independent problems make them unsuitable as reproduced performance results: (i) they used the same nonconstructive random timetable family whose routing core can already be infeasible, and (ii) the earlier driver retained variant labels but did not forward $d_{\max}$, $T_{\max}$, ν, or the paper-C13 toggle to the model. Longer runs cannot repair either experimental-design defect. Maintenance performance is therefore evaluated on the constructive and curated exact-feasible families in Table 12, while the original-versus-corrected C13 mechanism is tested with the genuine model toggle and targeted counterexample below. A numerical claim of reproducing Tables 10/11 requires the original timetables or a new generator that both guarantees routing feasibility and exposes every configured parameter.



(a) Route-pool size versus path time.

(b) Objective-equivalence check.

Figure 7: Scalability mechanism and correctness evidence for the path model.

7.14. Effect of the Constraint (13) correction

A caveat on earlier reported comparisons.. An earlier iteration of the experiment pipeline defined a `use_paper_c13` switch only at the configuration-file level (`experiments/configs/c13_correction.json`); it was never threaded through `src/model.py`'s model-building code, so every "paper-C13" run in that pipeline silently solved the corrected formulation. The results below use a genuine `use_paper_c13` flag that toggles between formulation (13_{paper}) and the corrected pair (13a)–(13b) inside the constraint builder itself, confirmed by the differing constraint counts the two variants produce on identical instances (Table 17).

Table 17 compares the two formulations on the existing targeted ABCD instances. The table is deliberately small and targeted: it exposes the mechanical difference between the two constraint formulations while showing that, on these two pre-existing test cases, the objective and status remain

unchanged. In other words, these instances are not tight enough (in flight-hour load relative to threshold, given the available fleet size and flat cost structure) to make the vacuity in constraint (13_{paper}) alter the optimal solution.

A targeted instance that exposes the bug.. Because the two instances above do not isolate the failure mode, we built a minimal instance designed directly from the counterexample in `docs/model_math.tex` (`data/instances/c13_loophole_test.json`): a single aircraft that must fly all 4 scheduled flights (350 min each, 1 400 min total) against a 1 000-minute A-check threshold, with maintenance capacity set to zero everywhere so no check can physically be performed. With C13b (the existing-hours constraint, which is unconditionally corrected-style and would otherwise mask the effect) disabled to isolate C13 alone:

- **Paper C13** (`use_paper_c13=True`): status *optimal*, objective 400, reporting a feasible schedule.
- **Corrected C13** (default): status *infeasible* – correctly rejecting a schedule that cannot legally fly 1 400 minutes under a 1 000-minute cap with zero maintenance opportunities.

We additionally built an independent, model-external validator (`src/validate_maintenance.py`) that recomputes cumulative flight-hours between actual check events directly from the solved `x`/`y` decision variables – without relying on the `mega` hierarchy variable or re-evaluating any big-M constraint expression. Applied to the "optimal" paper-C13 solution, the validator independently confirms a genuine violation: 1 400 minutes flown against a 1 000-

minute cap (400 minutes over), i.e. the paper formulation's "optimal" schedule is not actually maintenance-compliant. The corrected formulation raises this at solve time instead (infeasible), which is the intended fail-safe behavior.

Key findings:

- On the two pre-existing targeted ABCD instances, both formulations agree – these instances are not tight enough to expose the bug, so they should not be read as evidence that the correction has no computational effect.

- On a purpose-built instance directly instantiating the analytical counterexample, the two formulations diverge exactly as predicted: paper C13 reports a false "optimal" schedule that an independent validator confirms violates the flight-hour cap by 400 minutes, while corrected C13 correctly reports infeasibility.

7.15. Full hierarchy results

Table 18 reports the full $\{A, B, C, D\}$ hierarchy model with all constraint groups active and corrected C13 on the seven current `ABCD_*_test` instances. HiGHS is given 300 s per case. Four instances solve to optimality and three receive infeasibility certificates; none time out or terminate with an implementation error. The three infeasible files are retained as deliberate boundary tests, while performance claims are based on the exact-feasible rows and families.

7.16. Heuristic benchmark

Table 19 reports measured method values on `ABCD_cap_bottleneck` (8 flights, 3 aircraft). This instance is small enough to run the heuristics, exact-

search prototypes, and the MILP model side-by-side.

The heuristic-only comparison in Table 20 shows that the two deterministic constructive methods are effectively indistinguishable on this instance: both terminate in under a millisecond, cover seven of eight flights, and yield the same objective value. In contrast, the ACO heuristic explores a larger portion of the search space and achieves a lower partial-route cost, but this comes with reduced coverage, leaving three flights unassigned. The result is consistent with the draft's characterization of ACO as a more exploratory but less stable heuristic than the deterministic constructions.

Across all methods in Table 19, the MILP remains the only approach that returns a complete feasible assignment, and it does so with the best objective value on the benchmarked instance. The brute-force and DP exact-search prototypes remain useful as validation scaffolding, but on this instance they do not yet provide competitive production solutions for the full TAP search space.

7.17. Boundary suite and method coverage

Table 21 collects the larger boundary instances used to exercise the full method set. The suite covers the assignment bottleneck, check hierarchy, near-threshold maintenance, and mixed-maintenance cases. The constructive heuristics run on every instance; the exact-search prototypes are only applicable on the smallest case and degrade gracefully to `not_applicable` on the larger instances; and the MILP stays optimal on the first three cases while correctly reporting infeasibility on the `ABCD_two_b_one_c` boundary.

7.18. Heavy-instance stress test

The generated instances in `data/instances/` also include heavier cases with more flights and a longer planning horizon. These runs are useful for stress-testing the implementation under a realistic exact-solver backend.

On these larger instances, the two deterministic heuristics continue to return complete assignments quickly, while ACO remains more exploratory and leaves a larger residual objective. The exact-search prototypes are not applicable on the heavier cases because the feasible search space grows beyond their design target. With the full CPLEX Studio executable available, the MILP also solves all three of these medium-heavy instances to optimality, so the comparison on this suite becomes one of solution quality and runtime rather than solver availability.

7.19. Dense heavy instances

The densest generated cases push the MILP further than the lighter stress suite. They are useful for showing where the current solver environment stops being able to certify an exact solution, even though the heuristic methods still return schedules.

This dense suite sharpens the same qualitative conclusion as the lighter stress test. The constructive heuristics scale in a predictable way and keep producing valid schedules, albeit with larger residual objectives as the problem becomes denser and longer. By contrast, even with the full Studio backend, the exact MILP reaches the practical limit of the current environment on the denser instances: the smallest dense case is certified infeasible, whereas the larger horizons terminate without an exact certificate inside the current run budget.

7.20. Summary of findings

1. Our re-implementation faithfully reproduces the basic-model results of Tables 5 and 6 within the current exact-solver workflow; the CBC-backed run confirms the basic routing model is feasible and solves quickly on the $\delta = 0.5$ benchmark cells.
2. The corrected Constraint (13) is provably necessary (the original Khaled et al. formulation is vacuous whenever no check occurs at both window endpoints; Section 7.14) and a purpose-built instance confirms this in practice: the paper formulation reports a false "optimal" schedule that an independent validator shows violates the flight-hour cap, while the corrected formulation correctly reports infeasibility. On the two pre-existing targeted ABCD instances the two formulations happen to agree, so this should not be read as evidence that the correction never changes solutions in general.
3. The two deterministic heuristics behave almost identically on the bottleneck instance, suggesting that the current repair stage largely determines their final quality. In the fresh Phase-2 run, the repair variant reproduces the same objective values as the greedy-insertion heuristic on the five quick benchmark instances while keeping the same full-coverage behaviour on 24/25 cells and leaving one partial case.
4. ACO is more exploratory and can reduce the partial-route cost, but it also sacrifices coverage, which limits its usefulness as a stand-alone TAP solution method.
5. The MILP remains the strongest method when complete feasibility is required; on the small exact-feasible instances it improves the greedy

objective by roughly 77% while still solving in well under one second. On the larger random-grid benchmark cells, however, the local exact backend is currently blocked by solver-size limits before it can certify a feasible incumbent, so the benchmark should be read as a heuristic-coverage study on that family rather than as a complete exact-solver comparison.

6. The boundary suite confirms that all implemented methods execute on the key edge cases, with the exact-search prototypes degrading gracefully on larger instances and the MILP correctly distinguishing feasible from infeasible maintenance configurations.
7. The heavy-instance stress test shows that the heuristics remain fast on larger generated cases, and with the full CPLEX Studio executable the MILP also solves the medium-heavy density-0.5 instances to optimality.
8. The dense heavy cases confirm that this limitation becomes sharper as density and horizon increase: heuristics still solve the instances, but the exact MILP no longer certifies optimality on the densest horizons within the current run budget. The fresh CBC run reports optimality on the $\delta = 0.5$ cells and infeasibility on the denser ones, but the larger $|P| = 20$ or $|H| = 30$ cells hit the backend limit or abort before a full certificate is produced.
9. The arc-vs-path benchmark on the routing core (Section 7.12) shows objective equivalence on all seven existing ABCD tests but no consistent speed gain from exhaustive path-based enumeration; path-based is faster on only two cases and the arc-based formulation remains faster on average.

10. The brute-force and DP exact-search prototypes remain valuable for validation, but they are not yet competitive as production exact solvers for the full TAP search space.
11. The Phase-4 sensitivity sweep (Section 7.11) identifies maintenance thresholds as the most consistently binding parameter — they control feasibility on the tight side and cost on the loose side across the maintenance-active instances — while maintenance durations and station capacity bind only on specific topologies. With maintenance inactive, no parameter has any effect, confirming that the sensitivity is a genuine property of the integrated maintenance block.

8. Conclusions and Future Work

This paper revisited the compact Tail Assignment Problem model of Khaled et al. (2018), making four concrete contributions. The computational evidence in Section 7 is organised as a coherent report rather than a loose collection of experiments: the Phase-1 matrix and heatmap define the benchmark family, the Phase-2 tables contrast coverage and objective quality across the heuristic variants, the Phase-3 table and figure quantify the scalability boundary of the MILP, the Phase-4 sweep isolates the maintenance parameters that drive feasibility and runtime, the Phase-5 comparison contrasts the arc-based and path-based formulations, and the C13 comparison table isolates the correction effect in a targeted way.

C2–C4 feasibility correction.. We showed that the basic balance constraints can admit a simultaneous- departure corner case that is mathematically feasible but operationally impossible. Adding explicit pairwise non-overlap con-

straints restores physical executability without altering the variable set of the compact model. This correction also clarifies the domain on which maintenance constraints are evaluated.

Constraint (13) correction.. We identified and formally proved that the cumulative-flight-time constraint in the original paper admits vacuous satisfaction when only one of the two endpoint maintenance indicators is active. The corrected formulation uses two split constraints, each anchored to one endpoint, restoring validity, strictly tightening the LP relaxation, and requiring no additional variables. Computational experiments confirm that high-density instances with tight maintenance requirements yield different (and verifiably correct) solutions under the corrected model.

Effect on C8–C14.. Once C2–C4 are corrected, maintenance constraints C8–C14 do not require structural reformulation; instead, they operate on a stricter and physically valid assignment polytope. This removes spurious maintenance schedules that were artifacts of simultaneous infeasible flight assignments. Equivalently, Theorem 1 provides a formal corollary-level guarantee that any binary solution satisfying corrected C1–C14 is operationally executable.

Full $\{A, B, C, D\}$ hierarchy.. We extended the maintenance model from the single-type (type-A, flight-hour) framework to the full four-level hierarchy mandated by aviation regulators. The extension adds $O(n_f n_p n_d |\mathcal{C}|)$ constraints and variables and increases model solve times by a factor of approximately $2\times$, consistent with the trend observed for the type-A extension in the original paper. The hierarchical counter-reset logic and pre-horizon

accumulated hours/days are correctly handled.

Heuristic benchmark.. We provided the first systematic comparison of constructive heuristics against the compact MILP on the same instance family. The Phase-1 and Phase-2 results show that the deterministic heuristics remain coverage-stable over the 25-case suite, while the Phase-3 results show where the exact MILP becomes tractable and where the maintenance block overwhelms the solve time. The Dijkstra-based heuristic outperforms the ACO alternative in coverage on the temporal acyclic structure of the space-time network, and the greedy+insertion heuristic doubles as an effective warm-start provider for the MILP solver.

Reproducibility.. All code, instance generators, experiment configurations, and result tables are publicly available. The modular Pyomo implementation supports straightforward extension to new constraint families (e.g., crew connectivity, slot restrictions) via the constraint-toggle API.

Threats to validity.. The reported computational comparisons are intentionally scoped to instance-matched, objective-matched evaluations on the generated benchmark family used in this study. As a result, performance conclusions should be interpreted as within-scope (compact MILP plus the implemented heuristics) rather than universal rankings across all TAP paradigms. In particular, decomposition-heavy and robust/scenario alternatives may exhibit different trade-offs when evaluated under their native uncertainty assumptions, decomposition settings, and tuning protocols.

Future directions

- **Integrated fleet assignment and routing:** the compact model is well-suited for integration with fleet-assignment decisions, as noted by Khaled et al. (2018). The absence of the cyclic constraint facilitates this integration.

- **Robust and recoverable variants:** extending Borndörfer et al. (2010); Froyland et al. (2013) to the compact formulation with full maintenance hierarchy is an open problem.

- **Parallel branch-and-bound:** the compact model's very low branch-and-bound node counts (average < 15 for the basic model) suggest that warm-starting with multiple heuristic solutions in parallel could eliminate branching entirely on medium-sized instances.

- **Learning-based heuristics:** the greedy+insertion heuristic makes locally greedy decisions; reinforcement-learning or graph-neural- network approaches may improve solution quality without sacrificing speed.

References

Barnhart, C., Belobaba, P., Odoni, A.R., 2003. Applications of operations research in the air transport industry. Transportation Science 37, 368–391.

Barnhart, C., Boland, N.L., Clarke, L.W., Johnson, E.L., Nemhauser, G.L., Shenoi, R.G., 1998. Flight string models for aircraft fleeting and routing. Transportation Science 32, 208–220. doi:10.1287/trsc.32.3.208.

Borndörfer, R., Dovica, I., Nowak, I., Schickinger, T., 2010. Robust tail assignment. Technical Report. Konrad-Zuse-Zentrum für Informationstechnik Berlin.

Clarke, L., Johnson, E., Nemhauser, G., Zhu, Z., 1997. The aircraft rotation problem. Annals of Operations Research 69, 33–46. doi:10.1023/A:1018945415148.

Clarke, L.W., Hane, C.A., Johnson, E.L., Nemhauser, G.L., 1996. Maintenance and crew considerations in fleet assignment. Transportation Science 30, 249–260.

Cordeau, J.F., Stojković, G., Soumis, F., Desrosiers, J., 2001. Benders decomposition for simultaneous aircraft routing and crew scheduling. Transportation Science 35, 375–388.

Desaulniers, G., Desrosiers, J., Dumas, Y., Solomon, M.M., Soumis, F., 1997. Daily aircraft routing and scheduling. Management Science 43, 841–855.

Froyland, G., Maher, S.J., Wu, C.L., 2013. The recoverable robust tail assignment problem. Transportation Science 48, 351–372.

Gopalan, R., Talluri, K.T., 1998. Mathematical models in airline schedule planning: A survey. Annals of Operations Research 76, 155–185.

Grönkvist, M., 2005. The tail assignment problem. Ph.D. thesis. Chalmers, Göteborg University.

Hane, C.A., Barnhart, C., Johnson, E.L., Marsten, R.E., Nemhauser, G.L.,

Sigismondi, G., 1995. The fleet assignment problem: Solving a large-scale integer program. Mathematical Programming 70, 211–232.

Haouari, M., Shao, S., Sherali, H.D., 2012. A lifted compact formulation for the daily aircraft maintenance routing problem. Transportation Science 47, 508–525.

Hart, W.E., Laird, C.D., Watson, J.P., Woodruff, D.L., Hackebeil, G.A., Nicholson, B.L., Siirola, J.D., 2023. Pyomo – Python-based Optimization Modeling Objects. Version 6.x, `https://www.pyomo.org/`.

Khaled, O., Minoux, M., Mousseau, V., Michel, S., Ceugniet, X., 2018. A compact optimization model for the tail assignment problem. European Journal of Operational Research 264, 548–557. doi:`10.1016/j.ejor.2017.06.045`.

Klabjan, D., 2005. Large-scale models in the airline industry, in: Desaulniers, G., Desrosiers, J., Solomon, M.M. (Eds.), Column Generation. Springer US, pp. 163–195.

Lacasse-Guay, E., Desaulniers, G., Soumis, F., 2010. Aircraft routing under different business processes. Journal of Air Transport Management 16, 258–263.

Lavoie, S., Minoux, M., Odier, E., 1988. A new approach for crew pairing problems by column generation with an application to air transportation. European Journal of Operational Research 35, 45–58.

Levin, A., 1971. Scheduling and fleet routing models for transportation systems. Transportation Science 5, 232–255.

Liang, Z., Chaovalitwongse, W.A., 2012. A network-based model for the integrated weekly aircraft maintenance routing and fleet assignment problem. Transportation Science 47, 493–507.

Liang, Z., Chaovalitwongse, W.A., Huang, H.C., Johnson, E.L., 2011. On a new rotation tour network model for aircraft maintenance routing problem. Transportation Science 45, 109–120.

Mercier, A., Cordeau, J.F., Soumis, F., 2005. A computational study of Benders decomposition for the integrated aircraft routing and crew scheduling problem. Computers & Operations Research 32, 1451–1476. doi:10.1016/j.cor.2003.11.013.

Minoux, M., 1984. Column generation techniques in combinatorial optimization, a new application to crew pairing problems, in: Proceedings of the AGIFORS Symposium, Strasbourg, France.

Sarac, A., Batta, R., Rump, C.M., 2006. A branch-and-price approach for operational aircraft maintenance routing. European Journal of Operational Research 175, 1850–1869.

Sherali, H.D., Bish, E.K., Zhu, X., 2006. Airline fleet assignment concepts, models, and algorithms. European Journal of Operational Research 172, 1–30.

Table 7: Phase-1 summary statistics for the 25 canonical instances.

Instance	Density	Tier	Heuristic coverage	Category
I11	0.50	1	28/28	easy
I12	0.65	1	36/36	easy
I13	0.80	1	45/45	easy
I14	0.95	1	53/53	easy
I15	1.00	1	55/56	medium
I21	0.50	2	50/50	easy
I22	0.65	2	65/65	easy
I23	0.80	2	80/80	easy
I24	0.95	2	95/95	medium
I25	1.00	2	100/100	medium
I31	0.50	3	84/84	easy
I32	0.65	3	109/109	medium
I33	0.80	3	134/134	medium
I34	0.95	3	160/160	medium
I35	1.00	3	168/168	medium
I41	0.50	4	144/144	medium
I42	0.65	4	187/187	hard
I43	0.80	4	230/230	hard
I44	0.95	4	274/274	hard
I45	1.00	4	288/288	hard
I51	0.50	5	210/210	hard
I52	0.65	5	273/273	hard
I53	0.80	5	336/336	hard
I54	0.95	5	399/399	hard
I55	1.00	5	420/420	hard

Table 8: Phase-2 comparison on the 25-instance Phase-1 suite. The heuristic methods cover every instance and leave only one partially assigned case (I15, 55/56 flights). The MILP rows are not counted as solver-timeouts here; they are reported as solver-limit failures because the exact backend hits the size threshold before a certificate can be produced.

Method	Opt.	Inf.	Limit/err.	Heur.-assigned
Classical MILP	0	0	25	–
Integrated MILP	0	0	25	–
Greedy heuristic	–	–	–	25 (24 full, 1 partial)
Repair heuristic	–	–	–	25 (24 full, 1 partial)
Local-search heuristic	–	–	–	25 (24 full, 1 partial)

Table 9: Quick-objective snapshot for the five Phase-2 benchmark instances.

Instance	Greedy objective	Repair objective	Local-search objective
I11	117298	117298	105139
I12	144686	144686	108467
I13	164514	164514	158450
I14	242846	242846	188681
I15	196808	196808	172739

Table 10: Full Phase-2 benchmark summary over all 25 instances (CBC, heuristic methods only; MILP rows not shown because all 25 are `solver_limit`). Columns: instance identifier, fleet size $|P|$, horizon $|H|$, total flights $|F|$, greedy objective, local-search objective, relative improvement Δ (%), and local-search wall time (s).

Instance	$\lvert P\rvert$	$\lvert H\rvert$	$\lvert F\rvert$	Greedy obj.	LS obj.	Δ (%)	LS wall (s)
Tier 1 – easy ($p = 8$, $h = 7$)							
I11 ($\delta = 0.50$)	8	7	28	117 298	105 139	−10.4	2.0
I12 ($\delta = 0.65$)	8	7	36	144 686	108 467	−25.0	2.4
I13 ($\delta = 0.80$)	8	7	45	164 514	158 450	−3.7	1.8
I14 ($\delta = 0.95$)	8	7	53	242 846	188 681	−22.3	13.2
I15 ($\delta = 1.00$)	8	7	56	196 808	172 739	−12.2	4.4
Tier 2 – easy-medium ($p = 10$, $h = 10$)							
I21 ($\delta = 0.50$)	10	10	50	175 912	139 864	−20.5	3.4
I22 ($\delta = 0.65$)	10	10	65	320 225	235 946	−26.3	11.6
I23 ($\delta = 0.80$)	10	10	80	268 270	250 042	−6.8	5.6
I24 ($\delta = 0.95$)	10	10	95	399 934	345 603	−13.6	26.3
I25 ($\delta = 1.00$)	10	10	100	357 844	315 602	−11.8	14.1
Tier 3 – medium ($p = 12$, $h = 14$)							
I31 ($\delta = 0.50$)	12	14	84	310 726	262 338	−15.6	7.0
I32 ($\delta = 0.65$)	12	14	109	336 136	293 659	−12.6	13.5
I33 ($\delta = 0.80$)	12	14	134	429 968	387 399	−9.9	25.0
I34 ($\delta = 0.95$)	12	14	160	594 262	521 658	−12.2	56.1
I35 ($\delta = 1.00$)	12	14	168	550 873	490 476	−11.0	36.4
Tier 4 – medium-hard ($p = 16$, $h = 18$)							
I41 ($\delta = 0.50$)	16	18	144	495 609	434 586	−12.3	14.0
I42 ($\delta = 0.65$)	16	18	187	658 318	591 074	−10.2	44.1
I43 ($\delta = 0.80$)	16	18	230	716 243	649 381	−9.3	82.7
I44 ($\delta = 0.95$)	16	18	274	877 252	756 270	−13.8	176.0
I45 ($\delta = 1.00$)	16	18	288	900 073	809 290	−10.1	110.3
Tier 5 – hard ($p = 20$, $h = 21$)							

Table 11: Measured MILP benchmark on `ABCD_cap_bottleneck`. This compact row gives the solver statistics for one exact solve.

Status	Vars	Const	Gap (%)	CPU (s)	Obj
optimal	648	1 234	0.0000	0.27	8 300

Table 12: Exact-feasible instance comparison: Classical MILP (C1–C4, no maintenance), Integrated MILP (C1–C15 with maintenance), Greedy constructive heuristic, and Local-search refinement. Columns show optimal cost (Integrated MILP), heuristic objective, and gap relative to Integrated MILP optimal. Partial assignments (e.g. "7/8") indicate unassigned flights in the heuristic solution.

Instance	**p**	**f**	**Family**	**Clas. MILP**	**Int. MILP**	**Greedy**	**Gap (%)**	**LS**	**Gap (%)**
Constructive:									
`feas_A_p4h7_d05`	4	28	A-only	13027	17523	20513	+17.1	20513	+17.1
`feas_A_p4h7_d08`	4	30	A-only	13942	19032	22954	+20.6	22954	+20.6
`feas_A_p6h10_d05`	6	54	A-only	25100	34087	60121	+76.4	53624	+57.3
`feas_AB_p4h7_d05`	4	16	AB-mixed	7447	7847	11461	+46.1	11461	+46.1
`feas_ABCD_p4h7_d05`	4	8	ABCD	3728	4128	3728	–9.7	3728	–9.7
Curated:									
`ABCD_no_maint`	4	14	baseline	14000	14000	14000	0.0	14000	0.0
`ABCD_near_thr`	6	20	threshold	20000	21100	20000	–5.2	20000	–5.2
`ABCD_cap_botl`	3	8	capacity	8000	8300	37000†	+345.8	31000†	+273.5
`ABCD_hier`	4	13	hierarchy	13000	14700	35000†	+138.1	29000†	+97.3

Table 13: LP relaxation lower bounds for exact-feasible instances and one Phase-1 instance. Columns: Instance name, aircraft (p), flights (f), LP lower bound (Integrated formulation), Integrated MILP optimal (from Table 12), and integrality gap from LP bound to optimal MILP solution. The gap shows the maximum possible suboptimality gap if MILP were to hit a time limit: any heuristic within this bracket of MILP optimal is also within the bracket to the LP lower bound.

Instance	**p**	**f**	**Family**	**LP Bound**	**MILP Opt.**	**Integrality Gap (%)**
`ABCD_no_maint`	4	14	baseline	14,000	14,000	0.0
`ABCD_cap_botl`	3	8	capacity	8,000	8,300	3.8
`ABCD_hier`	4	13	hierarchy	13,000	14,700	13.1
`DataCplex_density=0.5_p=10_h=7`	10	64	random-grid	827,659	–	–

Table 14: Phase-3 model size and solve behaviour on the DataCplex benchmark (CBC, 120 s time limit). Variables are identical across both variants because they are declared unconditionally, while the constraint count and solve behaviour differ. "*inf*" denotes infeasible instances and "*err*" denotes CBC time-limit aborts.

				Classical MILP			Integrated MILP		
δ	$\lvert P\rvert$	$\lvert H\rvert$	Vars	Cons	Status	CPU(s)	Cons	Status	CPU(s)
0.5	10	7	14 540	3 582	opt	1.1	42 899	opt	48.6
0.5	10	15	37 590	3 080	opt	0.8	93 426	opt	47.0
0.5	10	21	37 790	1 879	opt	0.4	80 412	opt	33.2
0.95	10	7	10 320	1 686	inf	2.4	14 014	inf	4.8
0.95	10	15	32 560	3 892	inf	3.4	43 362	inf	26.0
0.95	20	7	39 280	10 433	inf	4.2	–	err	–
0.95	20	15	129 480	22 025	inf	25.5	–	err	–
1.0	10	7	10 480	1 870	inf	2.9	14 899	inf	4.7
1.0	10	15	32 760	4 080	inf	3.2	41 047	inf	25.0
1.0	10	21	60 620	5 840	inf	4.4	75 794	inf	79.3
1.0	10	30	–	–	err	–	–	err	–
1.0	20	7	39 320	11 940	inf	4.1	69 333	inf	85.1
1.0	20	15	133 440	24 780	inf	20.5	–	err	–
1.0	20	21	236 280	34 920	inf	54.1	–	err	–

Table 15: Phase-4 one-factor sensitivity sweep on the four curated exact-feasible instances (Integrated MILP, HiGHS, 120 s). Cells give the optimal objective at each scale multiple; `inf` marks a certified infeasible variant and "–" marks a scale not tested for that family (durations start at 50%; capacities are tested at 50/100/150/200%). The `no_maint` row-block is the control: with maintenance inactive, no parameter has any effect.

Parameter	Instance	25%	50%	75%	100%	150%	200%
Thresholds	`no_maint`	14000	14000	14000	14000	14000	14000
	`near_thr`	inf	inf	inf	21100	20000	20000
	`cap_botl`	inf	inf	inf	8300	8000	8000
	`hier`	14700	14700	14700	14700	13000	13000
Durations	`no_maint`	–	14000	14000	14000	14000	14000
	`near_thr`	–	21100	21100	21100	21100	21100
	`cap_botl`	–	8300	8300	8300	8300	8300
	`hier`	–	13300	13300	14700	inf	inf
Capacity	`no_maint`	–	14000	–	14000	14000	14000
	`near_thr`	–	21100	–	21100	21100	21100
	`cap_botl`	–	8300	–	8300	8300	8300
	`hier`	–	inf	–	14700	14700	14700

Table 16: Arc-based vs path-based MILP on the existing curated ABCD routing tests (no maintenance), HiGHS, ten repeats, 120 s limit. Times are end-to-end medians; "Speedup" is $T_{\text{arc}}/T_{\text{path}}$.

Instance	$\lvert F\rvert$	$\lvert P\rvert$	Arc obj	Path obj	Arc time (s)	Path time (s)	Arc vars	Path vars	Speedup
`ABCD_cap_botl`	8	3	8000	8000	0.0268	0.0197	24	135	1.360
`ABCD_all_checks`	9	5	9000	9000	0.0134	0.0176	9	106	0.737
`ABCD_check_hier`	13	4	13000	13000	0.0304	0.1087	52	920	0.277
`ABCD_no_maint`	14	4	14000	14000	0.0223	0.4858	56	3948	0.046
`ABCD_two_b_one_c`	14	5	14000	14000	0.0386	0.1164	70	1125	0.330
`ABCD_multi_check`	16	4	16000	16000	0.0132	0.3922	16	2592	0.033
`ABCD_near_thr`	20	6	20000	20000	0.0504	1.3059	120	6848	0.038
Median	–	–	–	–	0.0268	0.1164	–	–	0.277

Table 17: Constraint (13) comparison with the genuine `use_paper_c13` toggle. Constraint counts differ between variants (confirming the toggle is mechanically active), but neither targeted instance exposes a status or objective divergence.

Instance	Paper C13 obj.	Paper cons.	Corrected C13 obj.	Corr. cons.
`check_hierarchy`	14 700	2 277	14 700	2 445
`near_threshold`	21 100	5 028	21 100	5 280

Table 18: Full $\{A, B, C, D\}$ hierarchy results on the `ABCD_*_test` instance suite (HiGHS, 300 s time limit).

Instance	Status	Obj	Vars
`ABCD_all_checks_test`	infeasible	–	709
`ABCD_capacity_bottleneck_test`	optimal	8 300	648
`ABCD_check_hierarchy_test`	optimal	14 700	1 092
`ABCD_multi_check_test`	infeasible	–	808
`ABCD_near_threshold_test`	optimal	21 100	2 232
`ABCD_no_maint_test`	optimal	14 000	872
`ABCD_two_b_one_c_test`	infeasible	–	2 530

Table 19: Measured method values on `ABCD_cap_bottleneck`. Wall time is in seconds. The ACO objective is not directly comparable because it leaves flights unassigned.

Method	Status	Obj	Unassigned	Wall (s)
Greedy+Insertion	feasible	37 000	1	0.001
Dijkstra heuristic	feasible	37 000	1	0.001
ACO heuristic	feasible	17 000	3	0.086
Brute-force exact	infeasible	–	8	12.189
DP exact small	infeasible	–	8	12.251
MILP	optimal	8 300	0	0.203

Table 20: Heuristic-only comparison on `ABCD_capacity_bottleneck_test`. The table isolates the three constructive heuristics so their relative behavior is easier to read.

Method	Status	Obj	Unassigned
Greedy+Insertion	feasible	37 000	1
Dijkstra heuristic	feasible	37 000	1
ACO heuristic	feasible	17 000	3

Table 21: Boundary suite used to validate that every implemented method runs on the key model edges. Abbreviations: F = feasible, O = optimal, I = infeasible, N/A = not applicable.

Instance	Boundary	Gr.	Dijk.	ACO	BF	DP	MILP
`ABCD_capacity_bottleneck`	assignment bottleneck	F	F	F	I	I	O
`ABCD_check_hierarchy`	hierarchy	F	F	F	N/A	N/A	O
`ABCD_near_threshold`	hour threshold	F	F	F	N/A	N/A	O
`ABCD_two_b_one_c`	mixed maintenance	F	F	F	N/A	N/A	I

Table 22: Heavy-instance stress test on generated `DataCplex` cases with density $\delta = 0.5$ and $|P| = 10$. The exact-search prototypes are not applicable beyond the smallest case. With the full CPLEX Studio executable, the MILP solves all three reported instances to optimality.

Instance	Greedy	Dijkstra	ACO	BF exact	MILP
`h=7`	feasible / 0	feasible / 0	feasible / 80	not_applicable	optimal
`h=15`	feasible / 0	feasible / 0	feasible / 106	not_applicable	optimal
`h=21`	feasible / 0	feasible / 0	feasible / 79	not_applicable	optimal

Table 23: Dense heavy instances with density $\delta = 1.0$ and $|P| = 10$. These cases are substantially harder. The heuristic methods still produce schedules, while the MILP either proves infeasibility on the smallest dense case or stops without a certificate on the larger horizons.

Instance	Greedy	Dijkstra	ACO	MILP
`h=7`	feasible / 93	feasible / 119	feasible / 209	infeasible
`h=15`	feasible / 136	feasible / 278	feasible / 388	error
`h=21`	feasible / 149	feasible / 318	feasible / 461	error
`h=30`	feasible / 168	feasible / 330	feasible / 538	error